Alamein International University

Faculty of Computer Science & Engineering

CSE373 – Parallel Programming

Parallel Web Crawler

Project Reference

All Phases · What We Built · How Parallel Programming Is Applied

Team: [Ragnarok] | Course: CSE373

Phase1

Part 1: Proof of Applicability

Proof of Validity: Parallel Web Crawling for Threat Intelligence (OSINT)

1. Justification of the Idea: According to the project guidelines, teams are encouraged to propose alternative ideas that strictly apply parallel programming concepts and multi-stage pipelines. We propose a "Parallel Web Crawler for Open-Source Intelligence (OSINT)".

Web crawling is a fundamentally network I/O-bound task. Sequential crawling is highly inefficient because the CPU remains idle while waiting for HTTP responses. Applying parallel programming (Thread Pools) to fetch, parse, and store web data simultaneously is the industry standard for search engines and cybersecurity Threat Intelligence gathering.

2. Scientific Proof (Reference Paper): The validity of using Parallel Programming for Web Crawlers is heavily documented in computer science literature. The foundational scientific paper proving this is:

- **Paper Title:** *"Parallel Web Crawlers"*
- **Authors:** Junghoo Cho and Hector Garcia-Molina (Stanford University).
- **Published In:** Proceedings of the 11th International Conference on World Wide Web (WWW '02).
- **Relevance to this Project:** This paper mathematically proves that dividing the crawling workload across multiple parallel threads/processes drastically improves throughput and reduces download time. Our project implements the exact "Concurrent Fetching and Parsing" architecture discussed in this scientific paper, effectively replacing the suggested "Image Processing" pipeline with a real-world Data Extraction pipeline.

Part 2: System Requirements

1. Functional Requirements (What the system must do):

- **FR-01 (Configuration):** The user must be able to configure the starting URL (Seed), the maximum crawl depth, and the maximum number of pages.
- **FR-02 (Thread Management):** The user must be able to specify the number of parallel worker threads (from 1 to 20).
- **FR-03 (Parallel Fetching):** The system must download multiple HTML pages concurrently using a Thread Pool.
- **FR-04 (Data Extraction):** The system must extract the page title, meta description, HTTP status code, response time, and all outbound links from the HTML.
- **FR-05 (Concurrent Storage):** The system must securely save the extracted data into CSV, JSON, and SQLite database formats simultaneously.

- **FR-06 (Live Dashboard):** The system must provide a live JavaFX GUI to display crawling progress, success/error rates, and an active log.

2. Non-Functional Requirements (How well the system performs):

- **NFR-01 (Thread Safety & Synchronization):** The system must guarantee zero "Race Conditions". It must use concurrent collections (`ConcurrentHashMap`, `LinkedBlockingQueue`) and `synchronized` blocks to ensure data integrity when writing to the SQLite database.
- **NFR-02 (Performance/Speedup):** The parallel execution mode must demonstrate a significant time reduction (speedup) compared to the sequential mode, satisfying the mandatory benchmark requirement.
- **NFR-03 (Fault Tolerance):** If a network connection drops or a website times out, the specific thread must catch the `IOException`, apply a retry logic, and continue without crashing the entire application.

Part 3: System Analysis (تحليل النظام)

System Architecture & Pipeline Analysis

To satisfy the requirement of building a "multi-stage processing pipeline", our system is designed using a multi-threaded Producer-Consumer architecture. The pipeline consists of 4 main stages:

Stage 1: Initialization & Task Queuing (Producer)

- The system starts with a single Seed URL.
- It places this URL into a thread-safe `LinkedBlockingQueue`.
- A `ConcurrentHashMap` is used as a global "Visited Set" to ensure no URL is processed twice, acting as an atomic lock.

Stage 2: Parallel Fetching (I/O Bound)

- An `ExecutorService` (Fixed Thread Pool) is initialized with the user's defined thread count.
- Multiple worker threads simultaneously pull URLs from the queue and send HTTP GET requests (using Jsoup). This eliminates network latency bottlenecks.

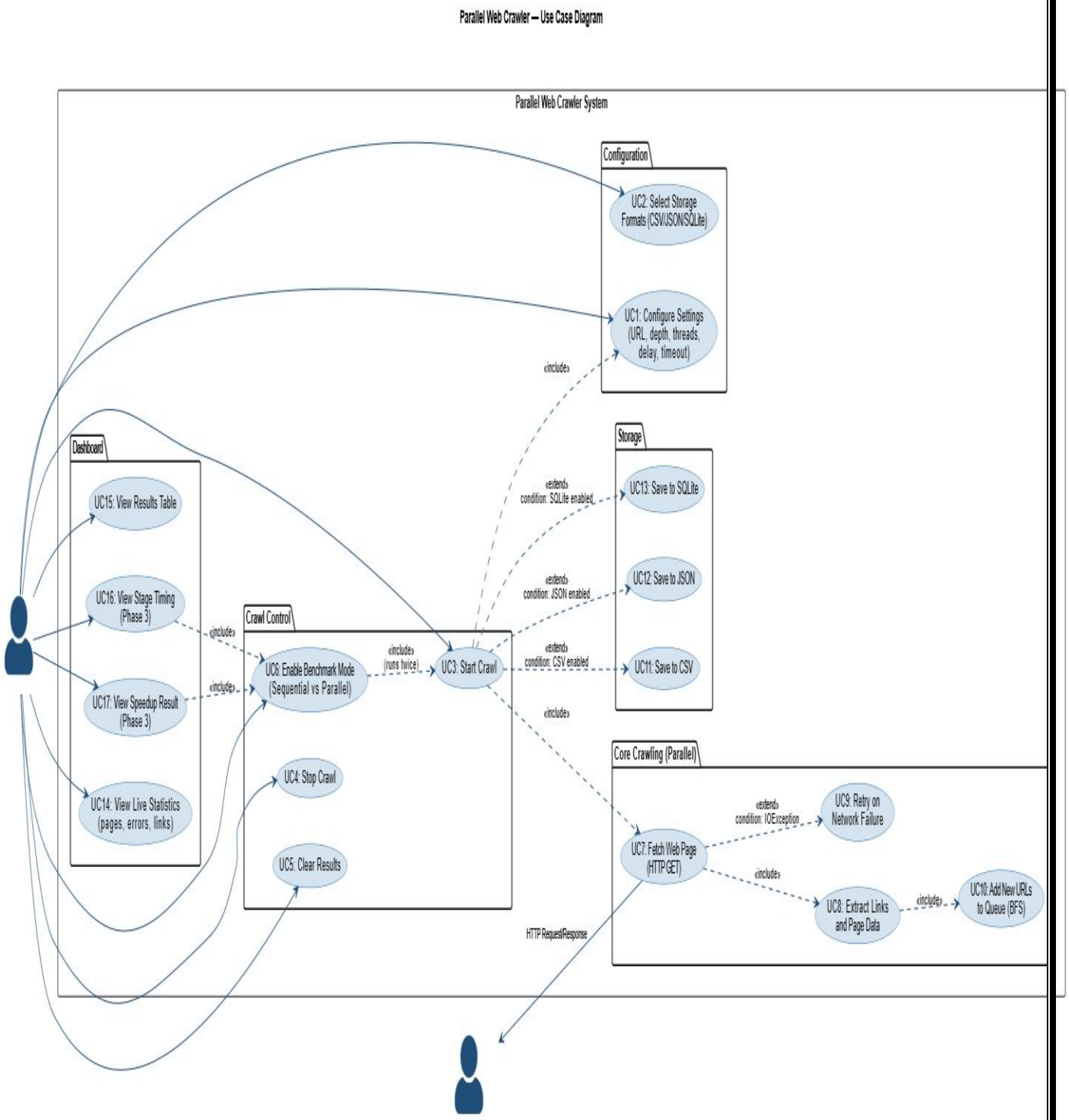
Stage 3: Concurrent Parsing (CPU Bound)

- Once a thread receives the HTML document, it parses the DOM tree to extract Metadata (Title, Description) and discovers new `<a href>` links.
- Valid new links are instantly fed back into the `LinkedBlockingQueue` for other idle threads to pick up.

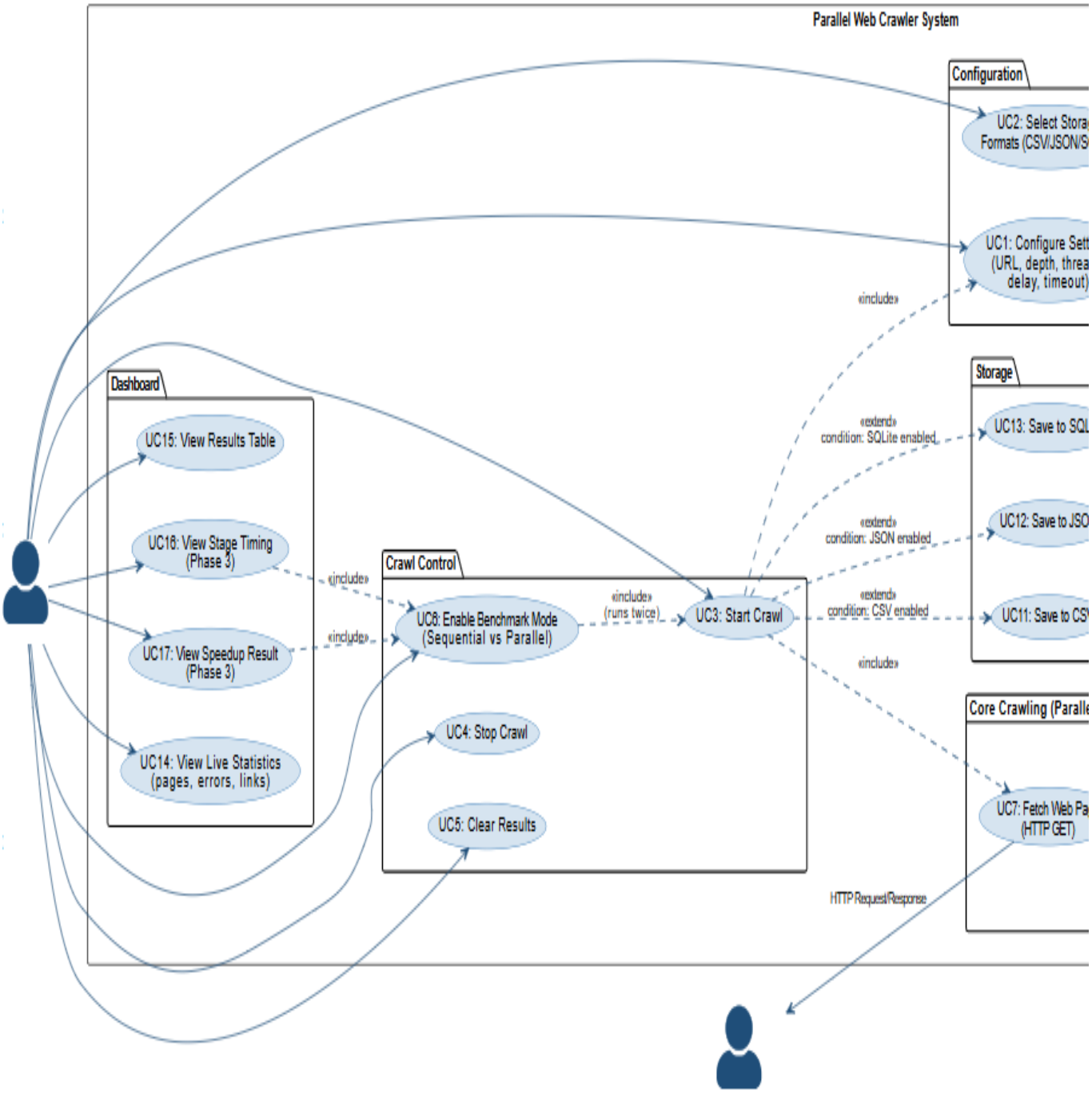
Stage 4: Synchronized Storage & Reporting

- Extracted page data is sent to the Storage Layer.
- Because multiple threads finish Stage 3 at the same time, writing to a single SQLite file or CSV would cause data corruption. We apply the `synchronized` keyword to our database writing functions, forcing threads to write sequentially to the disk while still downloading in parallel.
- Finally, the background threads use `Platform.runLater()` to safely send statistics to the JavaFX Application Thread, updating the live dashboard.

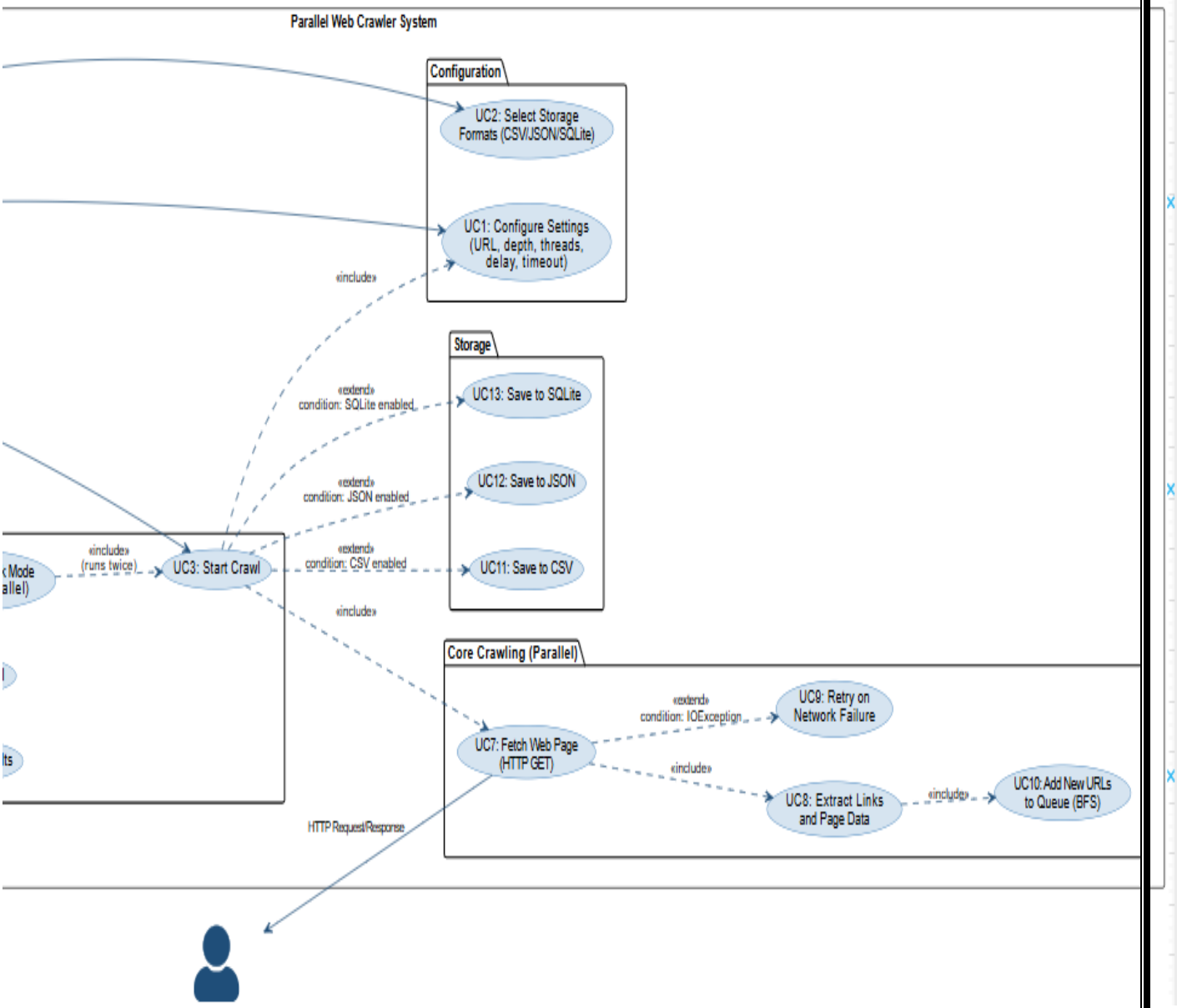
1. Use Case Diagram



Parallel Web Crawler — Use Case Diagram



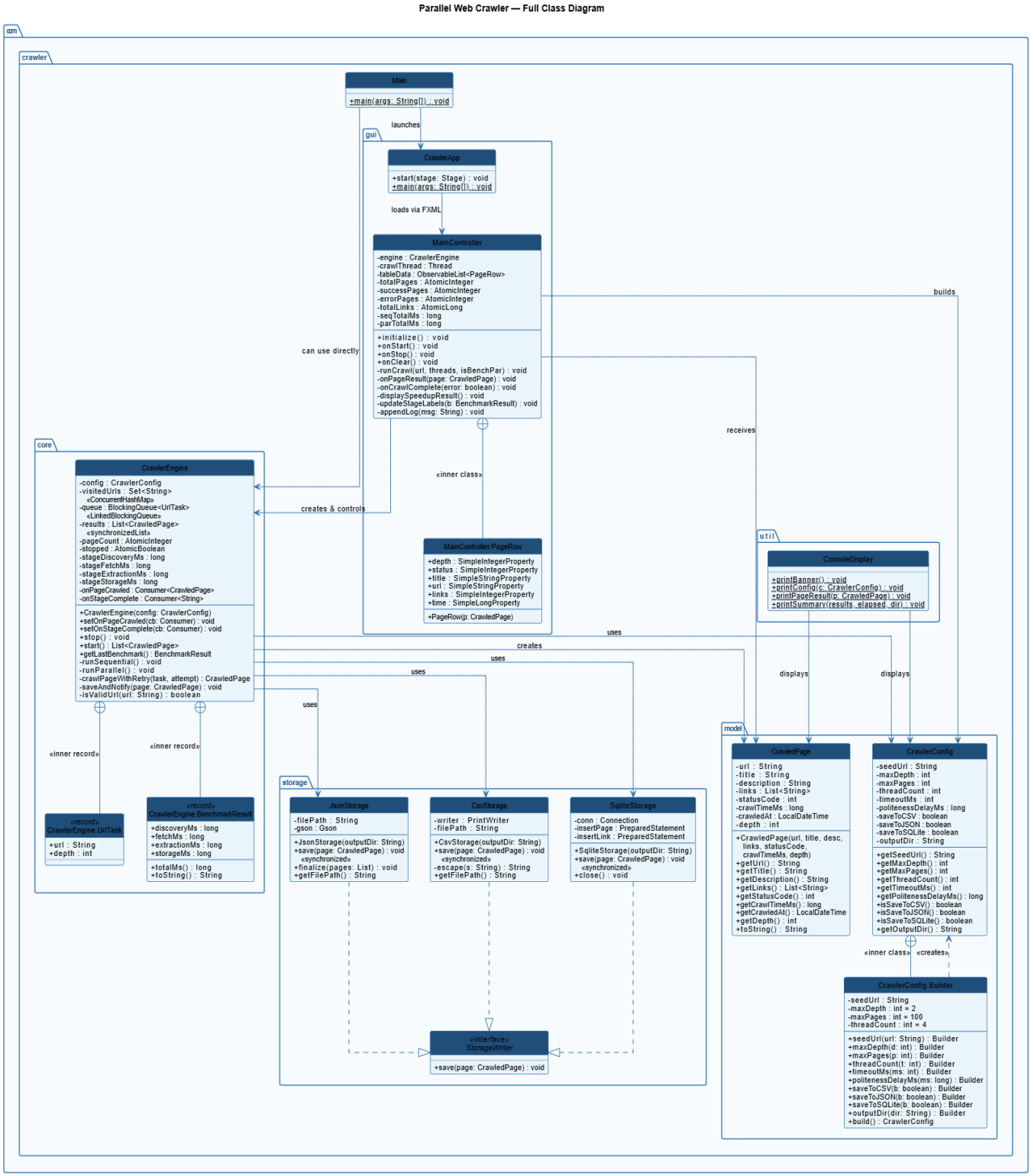
Parallel Web Crawler — Use Case Diagram

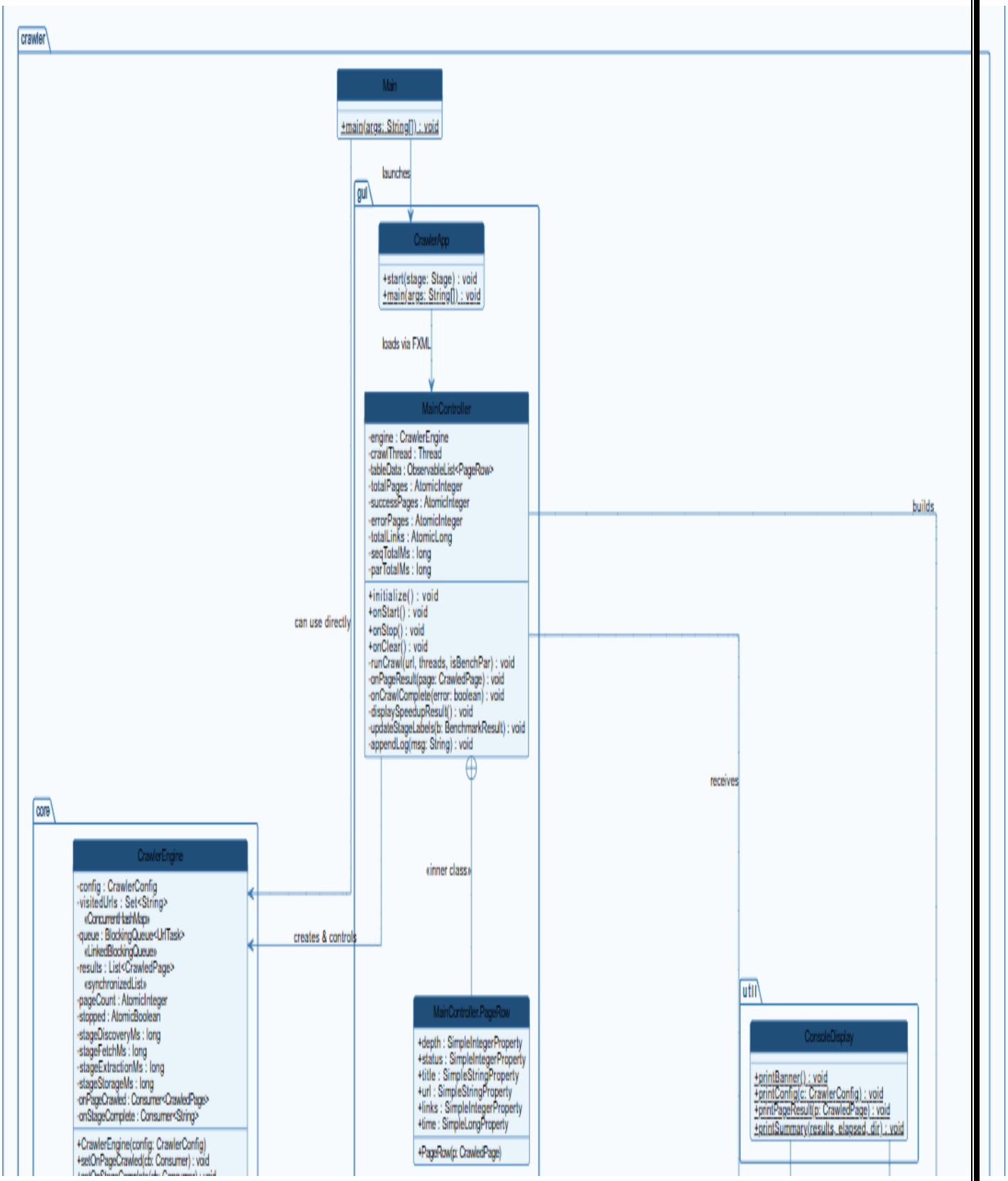


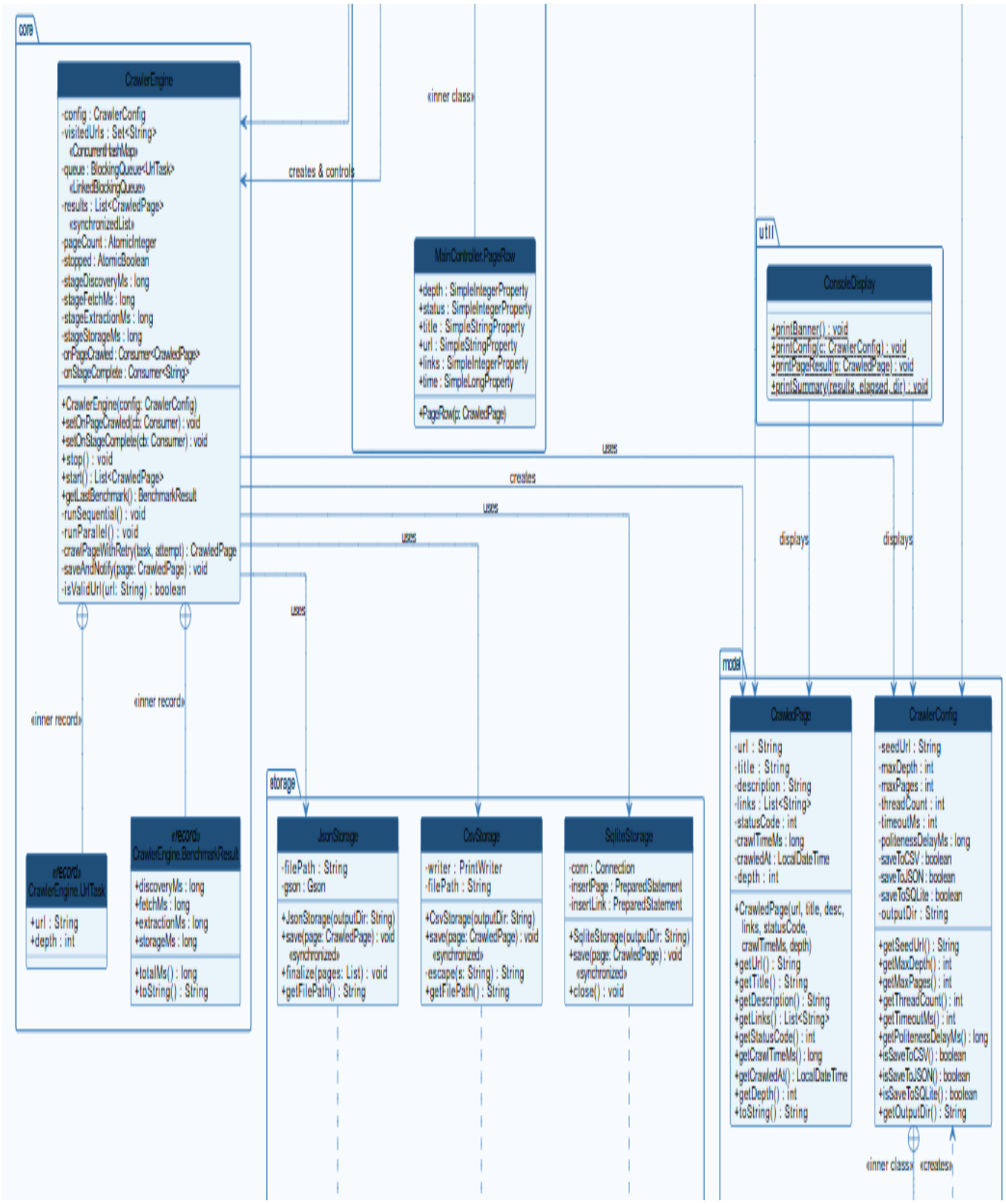
1. Use Case Diagram

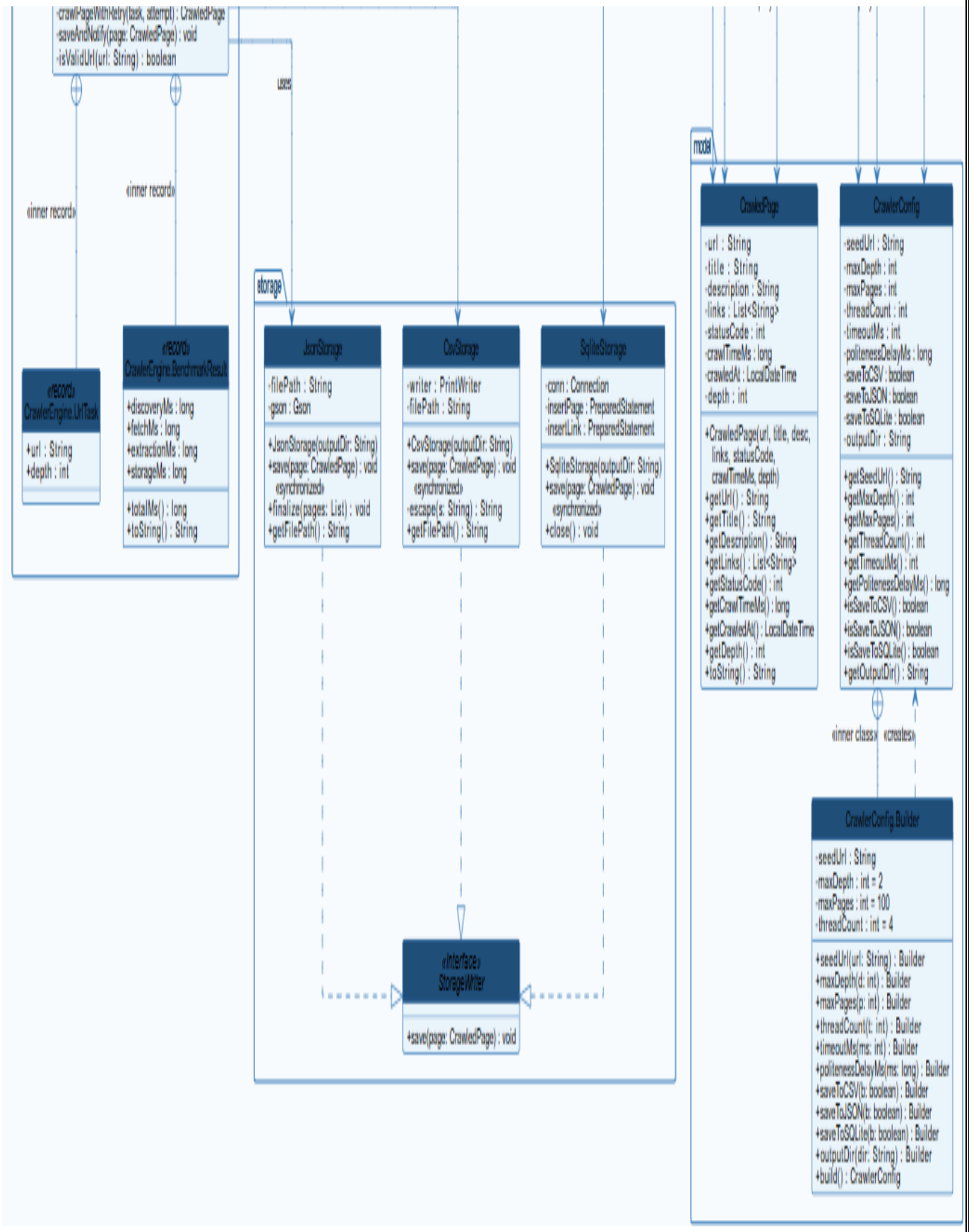
- **What it does:** It acts like a "menu of features". It shows the User (the actor) on one side, the Web Server on the other side, and all the actions the system can perform in the middle (like Start Crawl, Stop Crawl, View Stats, Export Data).
- **The Goal:** To show **WHAT** the system does from the user's perspective, without worrying about *how* the code actually works. It proves to the doctor that your system has all the required features.

2. Class Diagram




crawler
Main
+main(args: String[]) : void
launches
gui
CrawlerApp
+start(stage: Stage) : void
+main(args: String[]) : void
loads via FXML
MainController
-engine : CrawlerEngine
-crawlThread : Thread
-tableData : ObservableList<PageRow>
-totalPages : AtomicInteger
-successPages : AtomicInteger
-errorPages : AtomicInteger
-totalLinks : AtomicLong
-seqTotalMs : long
-parTotalMs : long
+initialize() : void
+onStart() : void
+onStop() : void
+onClear() : void
-runCrawl(url, threads, isBenchPar) : void
-onPageResult(page: CrawledPage) : void
-onCrawlComplete(error: boolean) : void
-displaySpeedupResult() : void
-updateStageLabels(b: BenchmarkResult) : void
-appendLog(msg: String) : void
builds
can use directly
receives
core
CrawlerEngine
-config : CrawlerConfig
-visitedUrls : Set<String>
«ConcurrentHashMap»
-queue : BlockingQueue<UrlTask>
«LinkedBlockingQueue»
-results : List<CrawledPage>
«synchronizedList»
-pageCount : AtomicInteger
-stopped : AtomicBoolean
-stageDiscoveryMs : long
-stageFetchMs : long
-stageExtractionMs : long
-stageStorageMs : long
-onPageCrawled : Consumer<CrawledPage>
-onStageComplete : Consumer<String>
+CrawlerEngine(config: CrawlerConfig)
+setOnPageCrawled(cb: Consumer) : void
«inner class»
creates & controls
MainController.PageRow
+depth : SimpleIntegerProperty
+status : SimpleIntegerProperty
+title : SimpleStringProperty
+url : SimpleStringProperty
+links : SimpleIntegerProperty
+time : SimpleLongProperty
+PageRow(p: CrawledPage)
util
ConsoleDisplay
+printBanner() : void
+printConfig(c: CrawlerConfig) : void
+printPageResult(p: CrawledPage) : void
+printSummary(results, elapsed, dir) : void

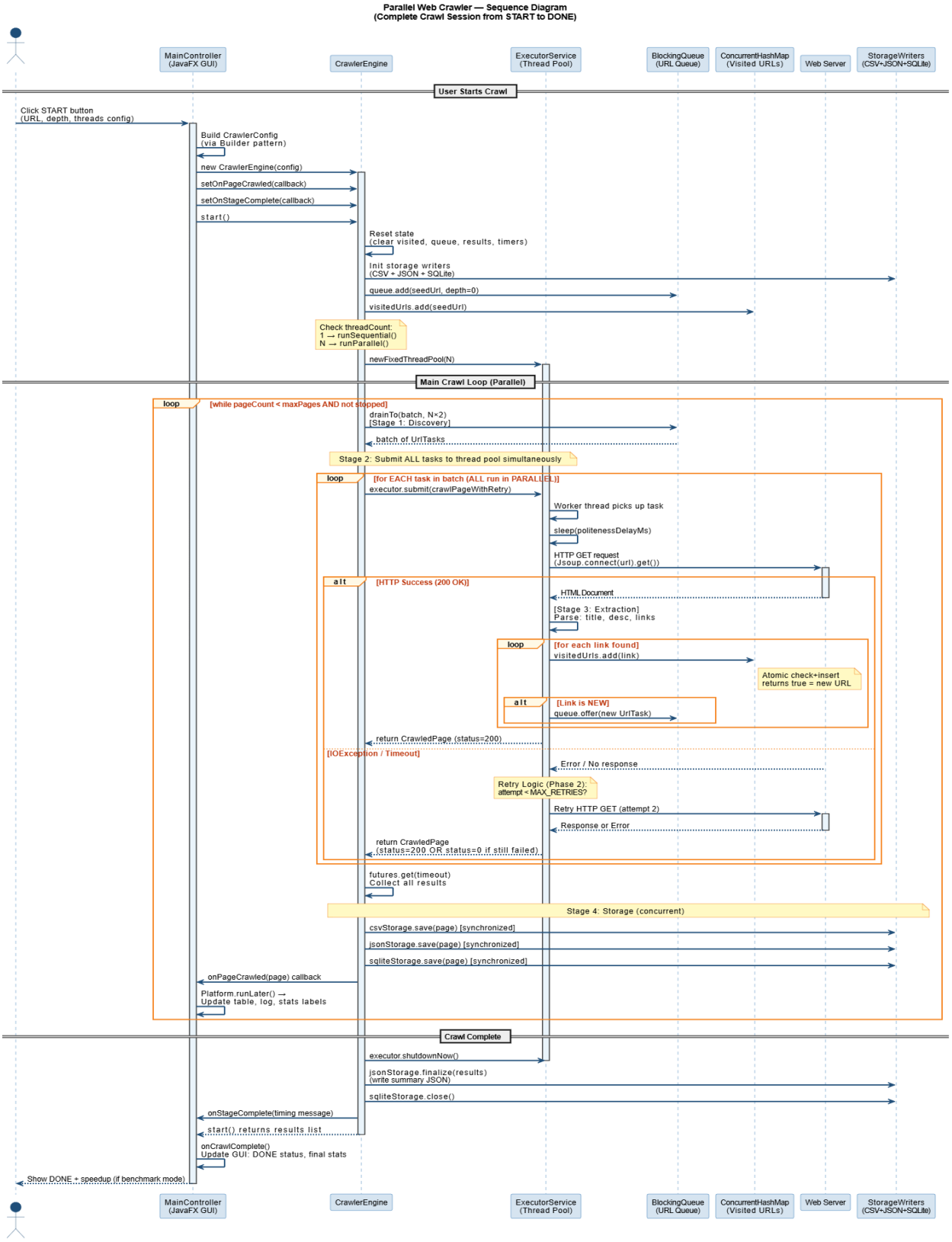

core
CrawlerEngine
-config : CrawlerConfig
-visitedUrls : Set<String>
«ConcurrentHashMap»
-queue : BlockingQueue<UrlTask>
«LinkedBlockingQueue»
-results : List<CrawledPage>
«synchronizedList»
-pageCount : AtomicInteger
-stopped : AtomicBoolean
-stageDiscoveryMs : long
-stageFetchMs : long
-stageExtractionMs : long
-stageStorageMs : long
-onPageCrawled : Consumer<CrawledPage>
-onStageComplete : Consumer<String>
+CrawlerEngine(config: CrawlerConfig)
+setOnPageCrawled(cb: Consumer) : void
+setOnStageComplete(cb: Consumer) : void
+stop() : void
+start() : List<CrawledPage>
+getLastBenchmark() : BenchmarkResult
-runSequential() : void
-runParallel() : void
-crawlPageWithRetry(task, attempt) : CrawledPage
-saveAndNotify(page: CrawledPage) : void
-isValidUrl(url: String) : boolean
«inner class»
MainController.PageRow
+depth : SimpleIntegerProperty
+status : SimpleIntegerProperty
+title : SimpleStringProperty
+url : SimpleStringProperty
+links : SimpleIntegerProperty
+time : SimpleLongProperty
+PageRow(p: CrawledPage)
creates & controls
util
ConsoleDisplay
+printBanner() : void
+printConfig(c: CrawlerConfig) : void
+printPageResult(p: CrawledPage) : void
+printSummary(results, elapsed, dir) : void
uses
creates
uses
uses
uses
displays
displays
«inner record»
«inner record»
«record»
CrawlerEngine.UrlTask
+url : String
+depth : int
«record»
CrawlerEngine.BenchmarkResult
+discoveryMs : long
+fetchMs : long
+extractionMs : long
+storageMs : long
+totalMs() : long
+toString() : String
storage
JsonStorage
-filePath : String
-gson : Gson
+JsonStorage(outputDir: String)
+save(page: CrawledPage) : void
«synchronized»
+finalize(pages: List) : void
+getFilePath() : String
CsvStorage
-writer : PrintWriter
-filePath : String
+CsvStorage(outputDir: String)
+save(page: CrawledPage) : void
«synchronized»
-escape(s: String) : String
+getFilePath() : String
SqliteStorage
-conn : Connection
-insertPage : PreparedStatement
-insertLink : PreparedStatement
+SqliteStorage(outputDir: String)
+save(page: CrawledPage) : void
«synchronized»
+close() : void
model
CrawledPage
-url : String
-title : String
-description : String
-links : List<String>
-statusCode : int
-crawlTimeMs : long
-crawledAt : LocalDateTime
-depth : int
+CrawledPage(url, title, desc, links, statusCode, crawlTimeMs, depth)
+getUrl() : String
+getTitle() : String
+getDescription() : String
+getLinks() : List<String>
+getStatusCode() : int
+getCrawlTimeMs() : long
+getCrawledAt() : LocalDateTime
+getDepth() : int
+toString() : String
CrawlerConfig
-seedUrl : String
-maxDepth : int
-maxPages : int
-threadCount : int
-timeoutMs : int
-politenessDelayMs : long
-saveToCSV : boolean
-saveToJSON : boolean
-saveToSQLite : boolean
-outputDir : String
+getSeedUrl() : String
+getMaxDepth() : int
+getMaxPages() : int
+getThreadCount() : int
+getTimeoutMs() : int
+getPolitenessDelayMs() : long
+isSaveToCSV() : boolean
+isSaveToJSON() : boolean
+isSaveToSQLite() : boolean
+getOutputDir() : String
«inner class»
«creates»

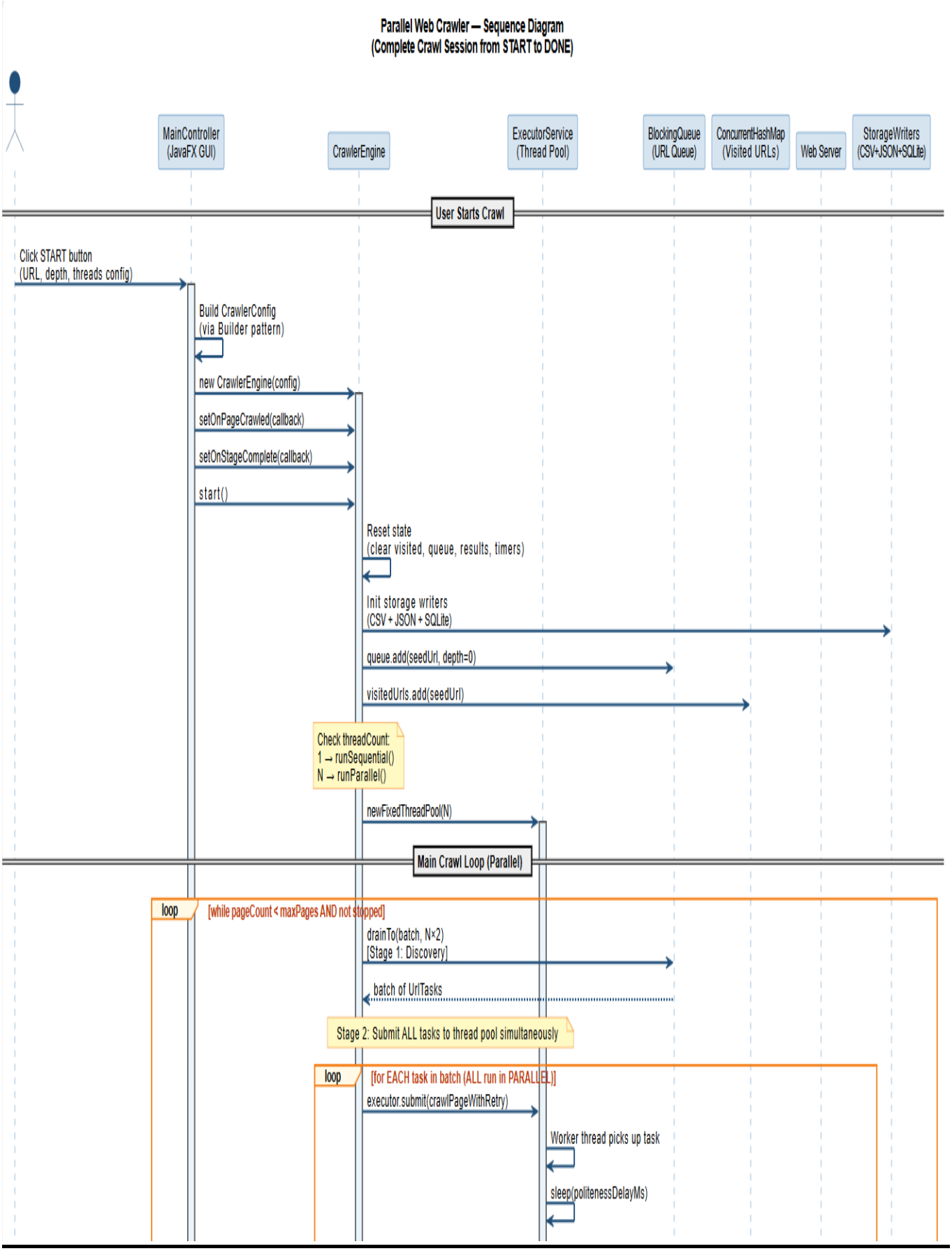

-crawlPageWithRetry(task, attempt) : CrawledPage
-saveAndNotify(page: CrawledPage) : void
-isValidUrl(url: String) : boolean
uses
«inner record»
«inner record»
«record»
CrawlerEngine.UrlTask
+url : String
+depth : int
«record»
CrawlerEngine.BenchmarkResult
+discoveryMs : long
+fetchMs : long
+extractionMs : long
+storageMs : long
+totalMs() : long
+toString() : String
storage
JsonStorage
-filePath : String
-gson : Gson
+JsonStorage(outputDir: String)
+save(page: CrawledPage) : void
«synchronized»
+finalize(pages: List) : void
+getFilePath() : String
CsvStorage
-writer : PrintWriter
-filePath : String
+CsvStorage(outputDir: String)
+save(page: CrawledPage) : void
«synchronized»
-escape(s: String) : String
+getFilePath() : String
SqliteStorage
-conn : Connection
-insertPage : PreparedStatement
-insertLink : PreparedStatement
+SqliteStorage(outputDir: String)
+save(page: CrawledPage) : void
«synchronized»
+close() : void
«interface»
StorageWriter
+save(page: CrawledPage) : void
model
CrawledPage
-url : String
-title : String
-description : String
-links : List<String>
-statusCode : int
-crawlTimeMs : long
-crawledAt : LocalDateTime
-depth : int
+CrawledPage(url, title, desc, links, statusCode, crawlTimeMs, depth)
+getUrl() : String
+getTitle() : String
+getDescription() : String
+getLinks() : List<String>
+getStatusCode() : int
+getCrawlTimeMs() : long
+getCrawledAt() : LocalDateTime
+getDepth() : int
+toString() : String
CrawlerConfig
-seedUrl : String
-maxDepth : int
-maxPages : int
-threadCount : int
-timeoutMs : int
-politenessDelayMs : long
-saveToCSV : boolean
-saveToJSON : boolean
-saveToSQLite : boolean
-outputDir : String
+getSeedUrl() : String
+getMaxDepth() : int
+getMaxPages() : int
+getThreadCount() : int
+getTimeoutMs() : int
+getPolitenessDelayMs() : long
+isSaveToCSV() : boolean
+isSaveToJSON() : boolean
+isSaveToSQLite() : boolean
+getOutputDir() : String
«inner class»
«creates»
CrawlerConfig.Builder
-seedUrl : String
-maxDepth : int = 2
-maxPages : int = 100
-threadCount : int = 4
+seedUrl(url: String) : Builder
+maxDepth(d: int) : Builder
+maxPages(p: int) : Builder
+threadCount(t: int) : Builder
+timeoutMs(ms: int) : Builder
+politenessDelayMs(ms: long) : Builder
+saveToCSV(b: boolean) : Builder
+saveToJSON(b: boolean) : Builder
+saveToSQLite(b: boolean) : Builder
+outputDir(dir: String) : Builder
+build() : CrawlerConfig

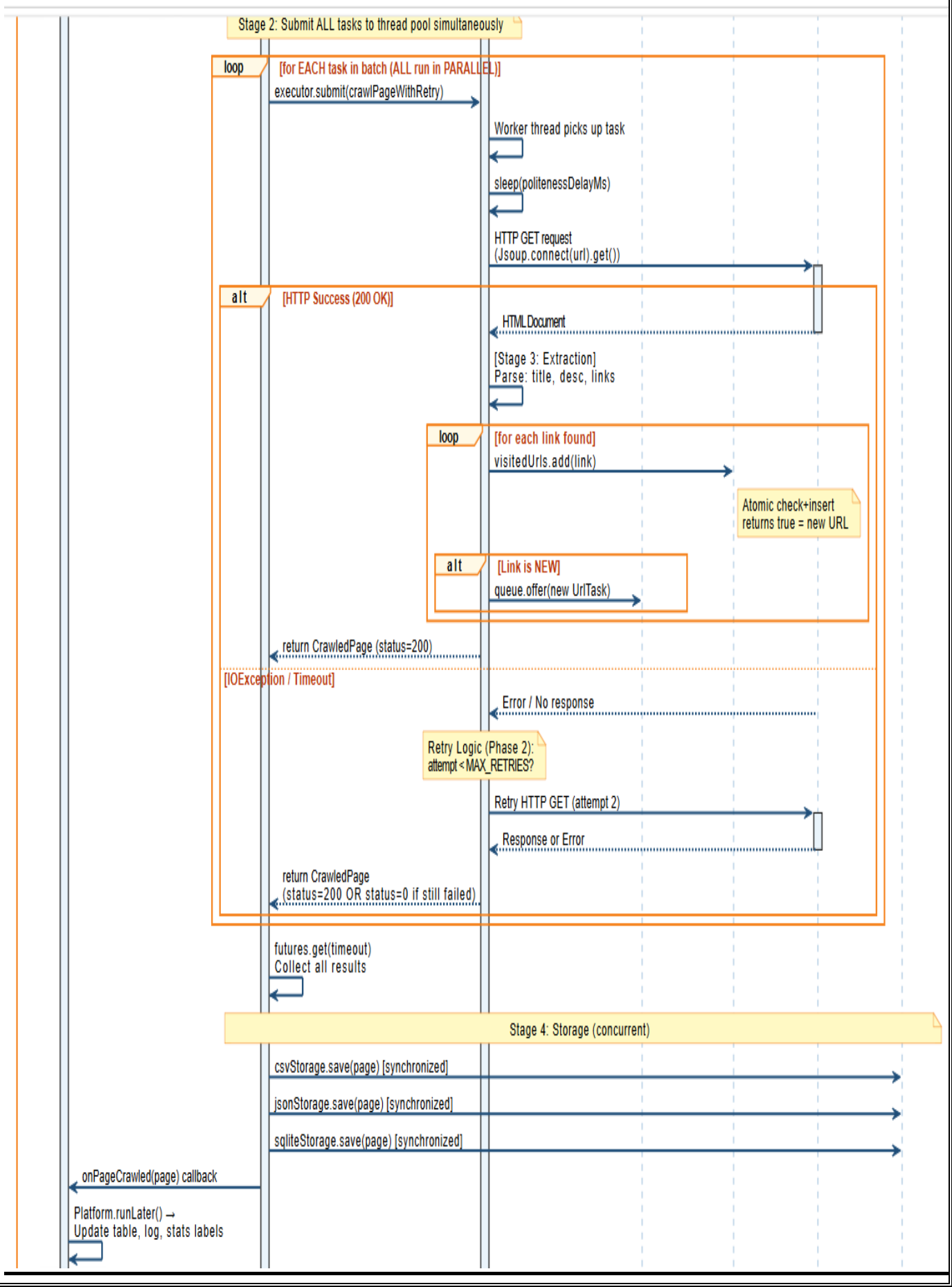
2. Class Diagram

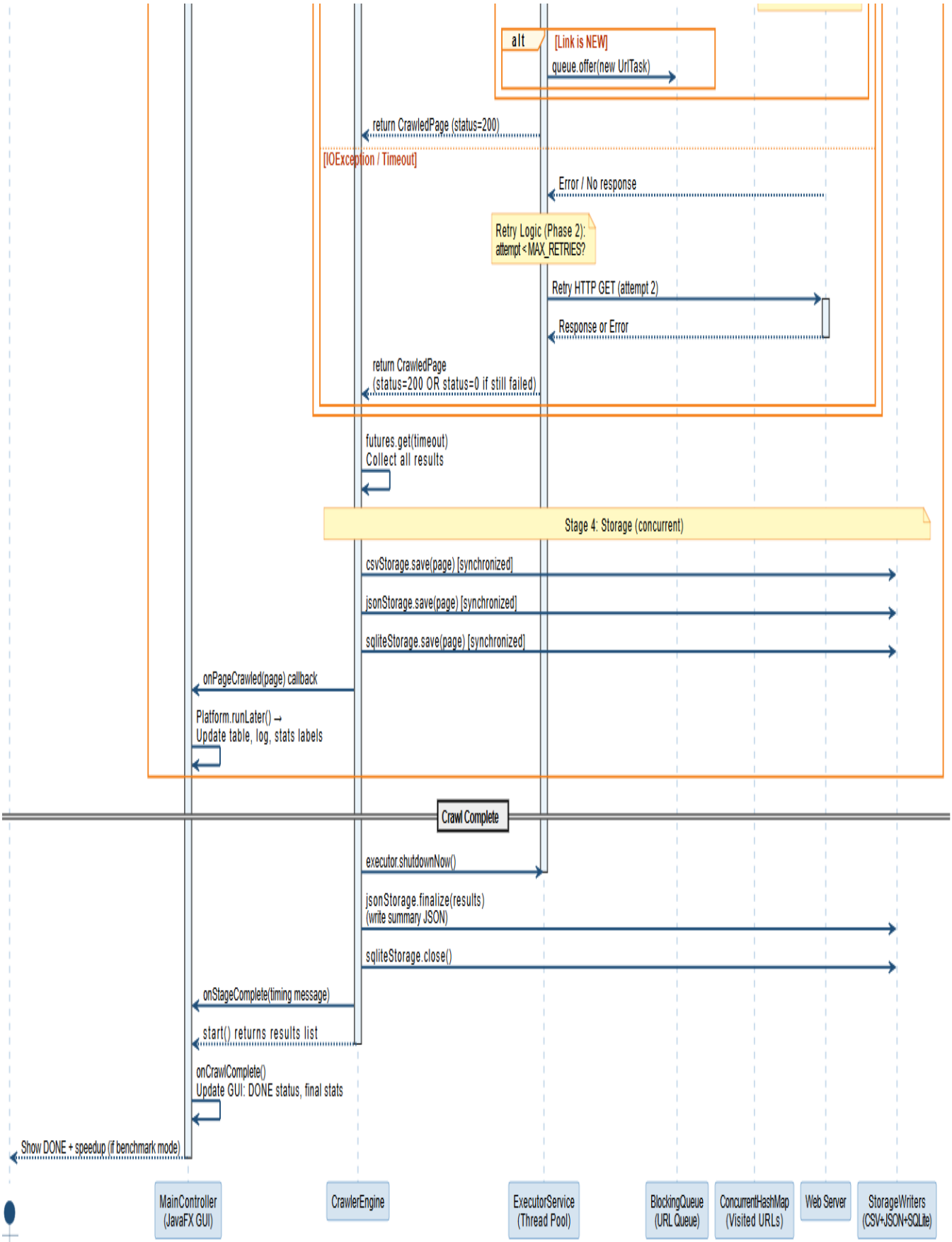
- **What it does:** It is the "blueprint" or the "map" of your Java code. It shows all the major Java files (Classes) like `CrawlerEngine`, `CrawlerConfig`, and `CrawledPage`. Inside every box, it lists the variables (data) and methods (functions) that belong to that class, and uses lines to show which class talks to which.
- **The Goal:** To show the **STRUCTURE** of your code. If another programmer looks at this, they will instantly know how you organized your project and how the pieces fit together.

3. Sequence Diagram




Parallel Web Crawler — Sequence Diagram
(Complete Crawl Session from START to DONE)
MainController (JavaFX GUI)
CrawlerEngine
ExecutorService (Thread Pool)
BlockingQueue (URL Queue)
ConcurrentHashMap (Visited URLs)
Web Server
StorageWriters (CSV+JSON+SQLite)
User Starts Crawl
Click START button (URL, depth, threads config)
Build CrawlerConfig (via Builder pattern)
new CrawlerEngine(config)
setOnPageCrawled(callback)
setOnStageComplete(callback)
start()
Reset state (clear visited, queue, results, timers)
Init storage writers (CSV + JSON + SQLite)
queue.add(seedUrl, depth=0)
visitedUrls.add(seedUrl)
Check threadCount: 1 → runSequential() N → runParallel()
newFixedThreadPool(N)
Main Crawl Loop (Parallel)
loop
[while pageCount < maxPages AND not stopped]
drainTo(batch, N×2) [Stage 1: Discovery]
batch of UrlTasks
Stage 2: Submit ALL tasks to thread pool simultaneously
loop
[for EACH task in batch (ALL run in PARALLEL)]
executor.submit(crawlPageWithRetry)
Worker thread picks up task
sleep(politenessDelayMs)

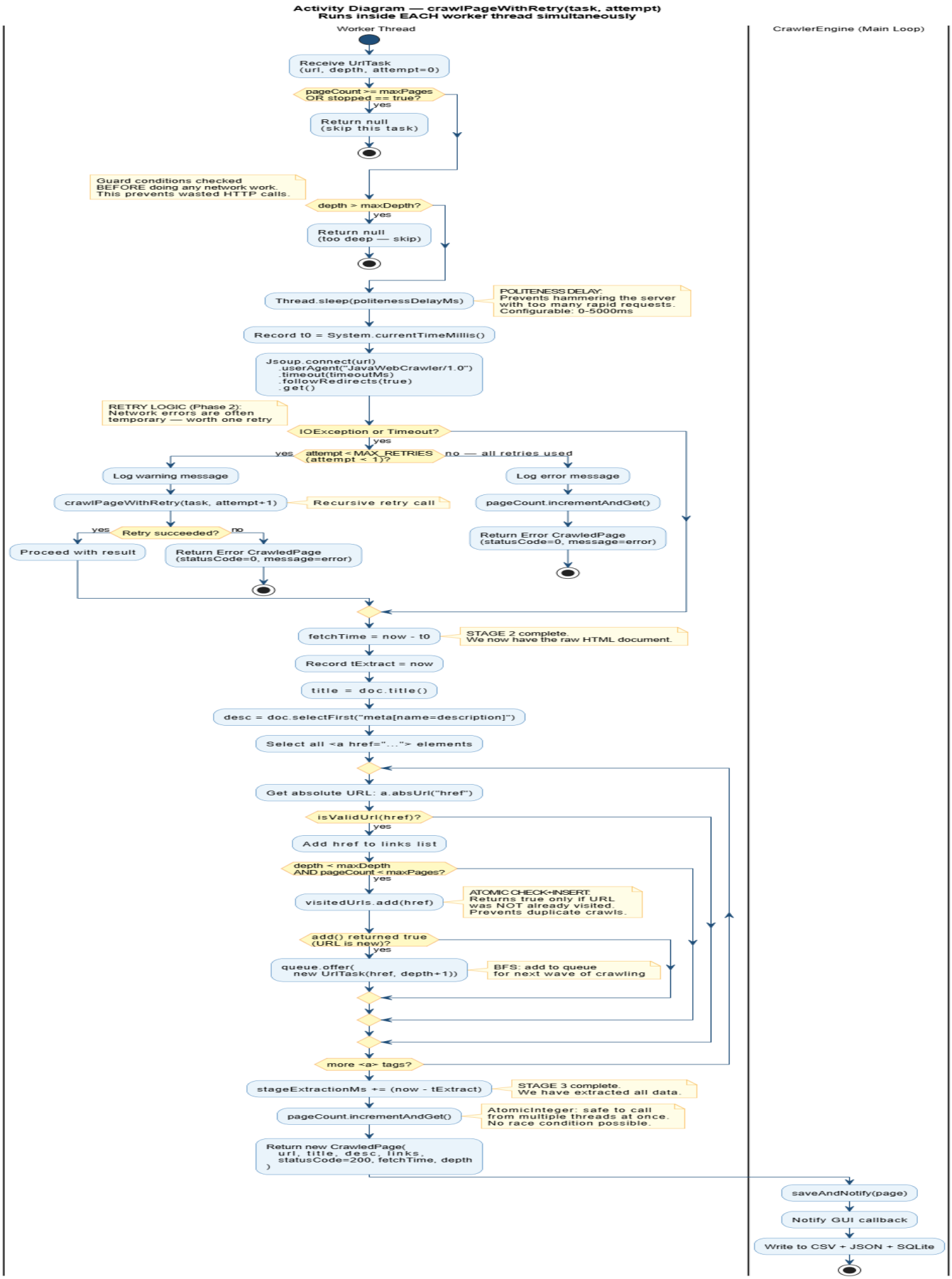

Stage 2: Submit ALL tasks to thread pool simultaneously
loop
[for EACH task in batch (ALL run in PARALLEL)]
executor.submit(crawlPageWithRetry)
Worker thread picks up task
sleep(politenessDelayMs)
HTTP GET request
(Jsoup.connect(url).get())
alt
[HTTP Success (200 OK)]
HTML Document
[Stage 3: Extraction]
Parse: title, desc, links
loop
[for each link found]
visitedUrls.add(link)
Atomic check+insert
returns true = new URL
alt
[Link is NEW]
queue.offer(new UrlTask)
return CrawledPage (status=200)
[IOException / Timeout]
Error / No response
Retry Logic (Phase 2):
attempt < MAX_RETRIES?
Retry HTTP GET (attempt 2)
Response or Error
return CrawledPage
(status=200 OR status=0 if still failed)
futures.get(timeout)
Collect all results
Stage 4: Storage (concurrent)
csvStorage.save(page) [synchronized]
jsonStorage.save(page) [synchronized]
sqliteStorage.save(page) [synchronized]
onPageCrawled(page) callback
Platform.runLater() →
Update table, log, stats labels


alt
[Link is NEW]
queue.offer(new UrlTask)
return CrawledPage (status=200)
[IOException / Timeout]
Error / No response
Retry Logic (Phase 2):
attempt < MAX_RETRIES?
Retry HTTP GET (attempt 2)
Response or Error
return CrawledPage
(status=200 OR status=0 if still failed)
futures.get(timeout)
Collect all results
Stage 4: Storage (concurrent)
csvStorage.save(page) [synchronized]
jsonStorage.save(page) [synchronized]
sqliteStorage.save(page) [synchronized]
onPageCrawled(page) callback
Platform.runLater() →
Update table, log, stats labels
Crawl Complete
executor.shutdownNow()
jsonStorage.finalize(results)
(write summary JSON)
sqliteStorage.close()
onStageComplete(timing message)
start() returns results list
onCrawlComplete()
Update GUI: DONE status, final stats
Show DONE + speedup (if benchmark mode)
MainController
(JavaFX GUI)
CrawlerEngine
ExecutorService
(Thread Pool)
BlockingQueue
(URL Queue)
ConcurrentHashMap
(Visited URLs)
Web Server
StorageWriters
(CSV+JSON+SQLite)

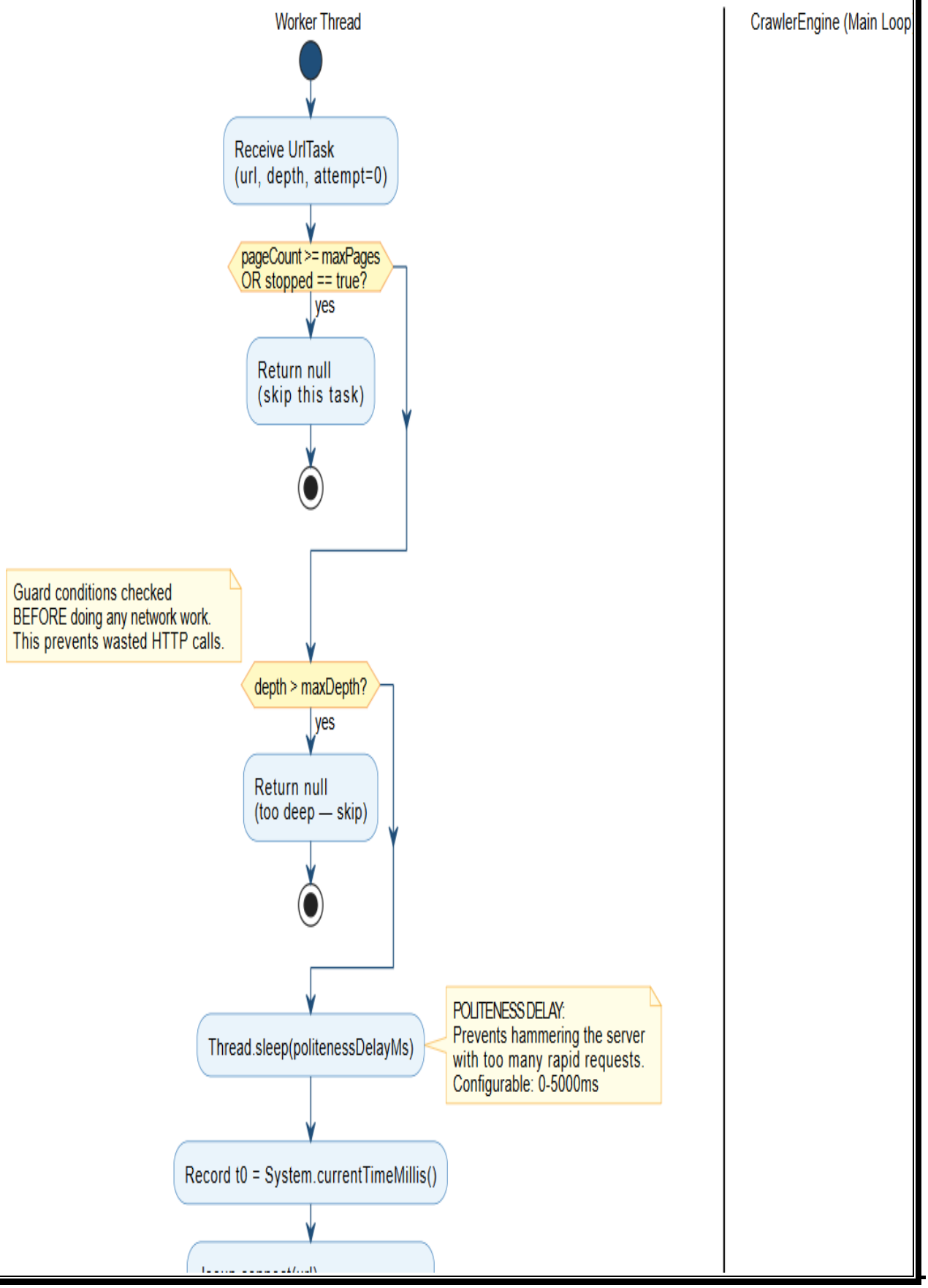
3. Sequence Diagram

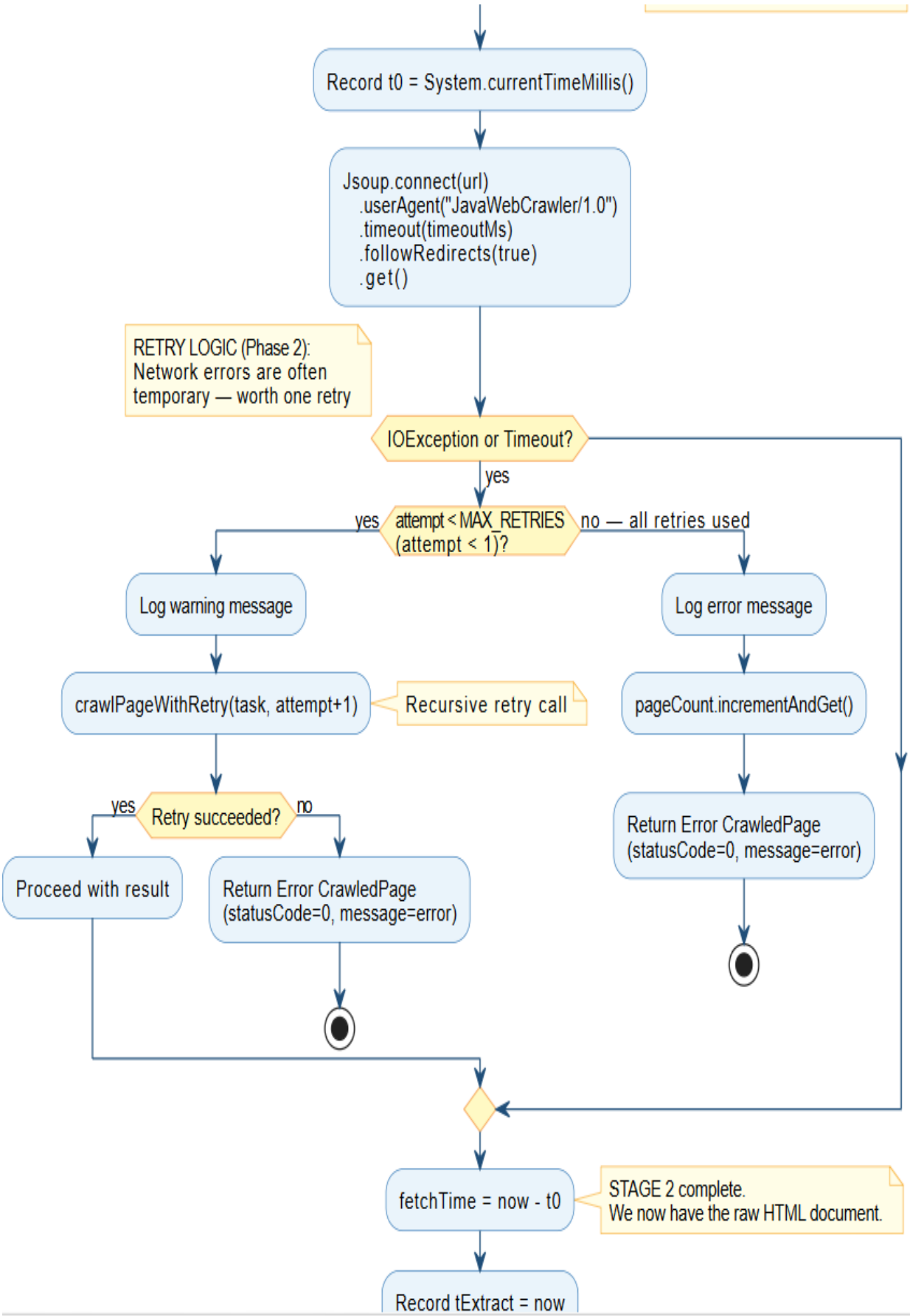
- **What it does:** It tells a story over time. It shows exactly what happens step-by-step from the moment the user clicks the "START" button until the program finishes. It shows how the User Interface talks to the Engine, how the Engine creates the Thread Pool, and how the threads talk to the Web Server and the Database.
- **The Goal:** To explain the **FLOW OF COMMUNICATION**. It proves that you understand the exact order of operations, especially how the parallel threads are created and managed during the crawl session.

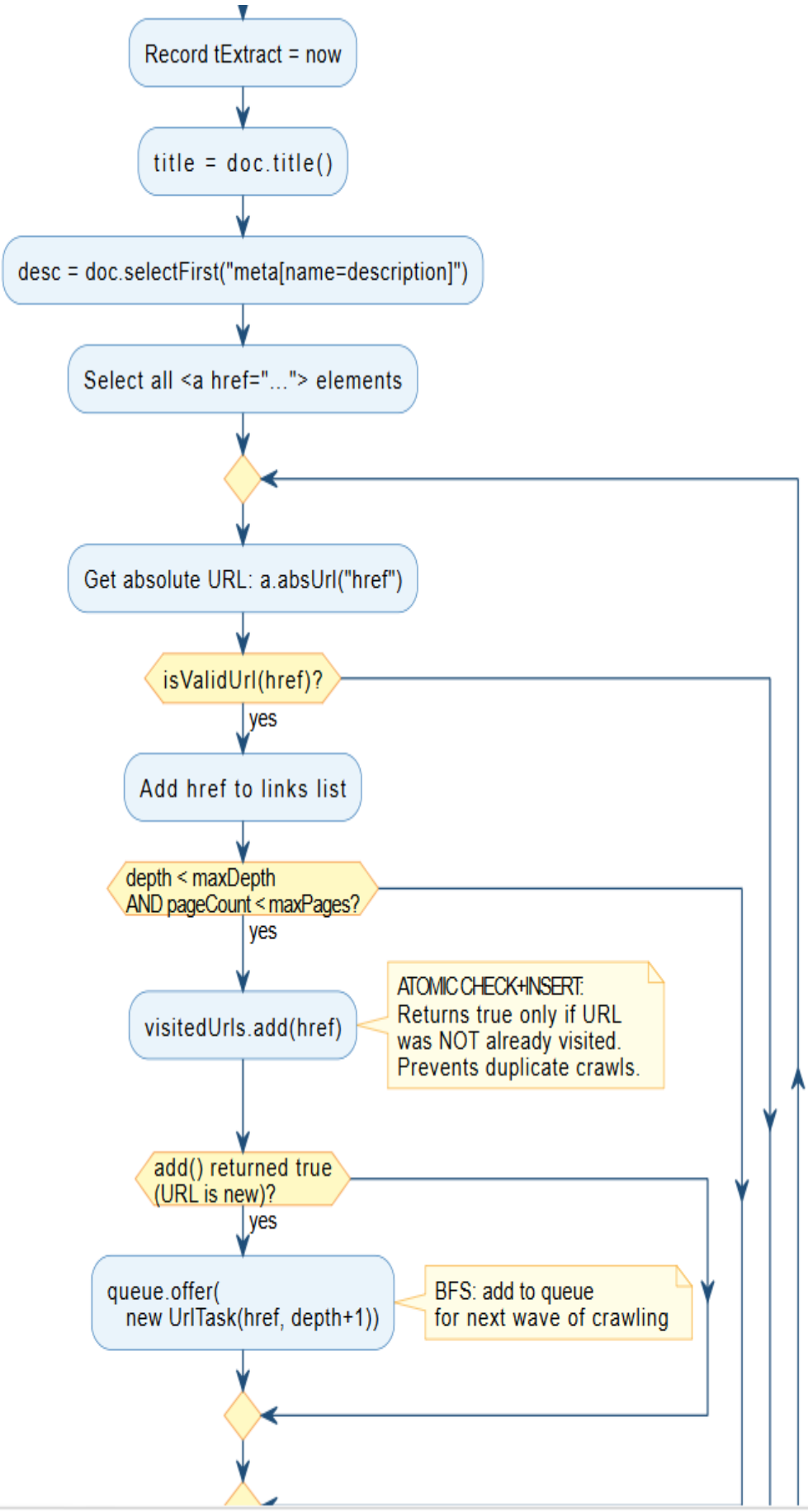
4. Activity Diagram

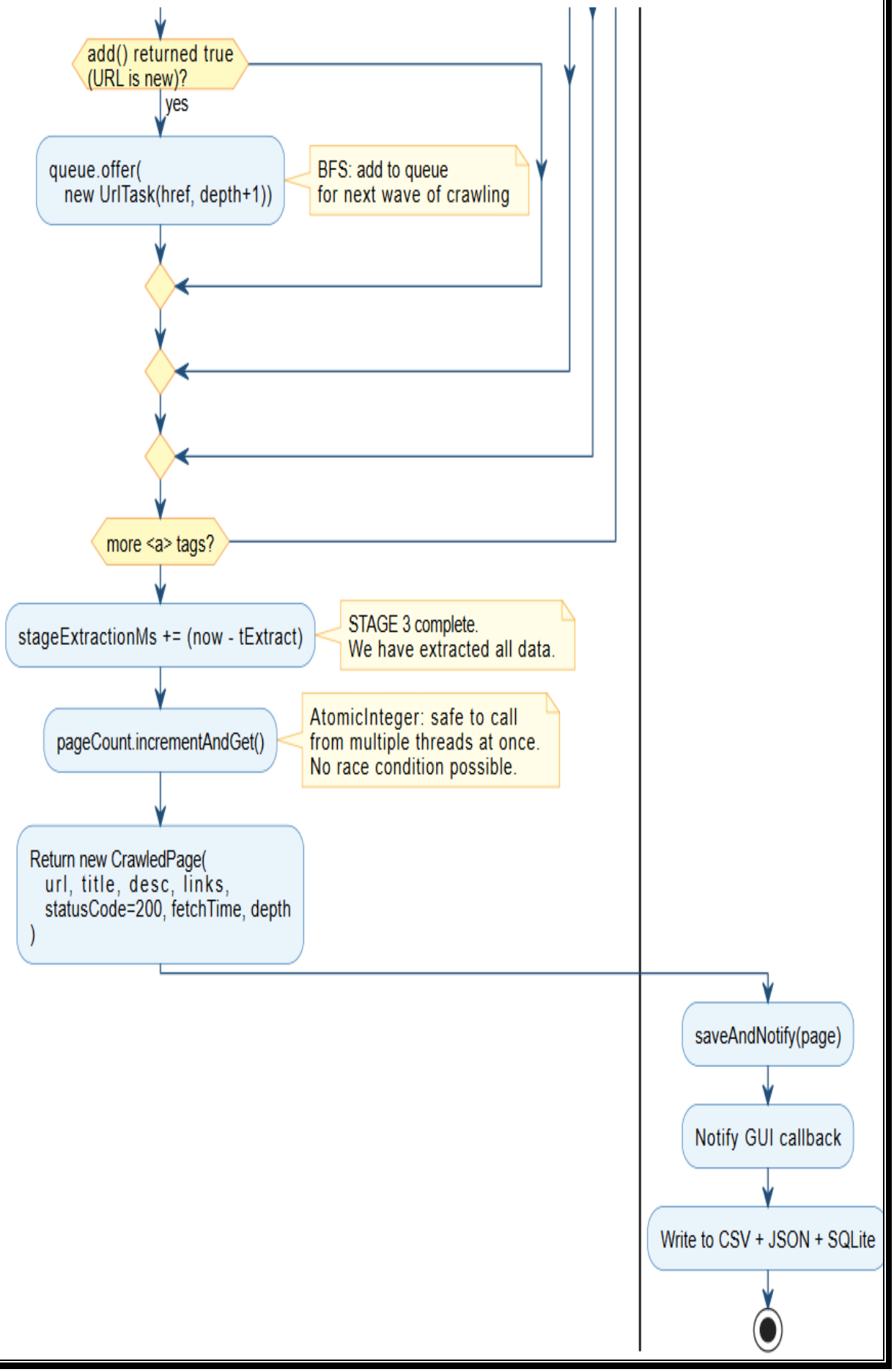

Activity Diagram — crawlPageWithRetry(task, attempt)
Runs inside EACH worker thread simultaneously
Worker Thread
CrawlerEngine (Main Loop)
Receive UrlTask (url, depth, attempt=0)
pageCount >= maxPages OR stopped == true?
yes
Return null (skip this task)
Guard conditions checked BEFORE doing any network work. This prevents wasted HTTP calls.
depth > maxDepth?
yes
Return null (too deep — skip)
Thread.sleep(politenessDelayMs)
POLITENESS DELAY: Prevents hammering the server with too many rapid requests. Configurable: 0-5000ms
Record t0 = System.currentTimeMillis()
Jsoup.connect(url) .userAgent("JavaWebCrawler/1.0") .timeout(timeoutMs) .followRedirects(true) .get()
RETRY LOGIC (Phase 2): Network errors are often temporary — worth one retry
IOException or Timeout?
yes
attempt < MAX_RETRIES (attempt < 1)?
yes
no — all retries used
Log warning message
Log error message
crawlPageWithRetry(task, attempt+1)
Recursive retry call
pageCount.incrementAndGet()
Retry succeeded?
yes
no
Proceed with result
Return Error CrawledPage (statusCode=0, message=error)
Return Error CrawledPage (statusCode=0, message=error)
fetchTime = now - t0
STAGE 2 complete. We now have the raw HTML document.
Record tExtract = now
title = doc.title()
desc = doc.selectFirst("meta[name=description]")
Select all <a href="..."> elements
Get absolute URL: a.absUrl("href")
isValidUrl(href)?
yes
Add href to links list
depth < maxDepth AND pageCount < maxPages?
yes
visitedUrls.add(href)
ATOMIC CHECK+INSERT: Returns true only if URL was NOT already visited. Prevents duplicate crawls.
add() returned true (URL is new)?
yes
queue.offer(new UrlTask(href, depth+1))
BFS: add to queue for next wave of crawling
more <a> tags?
stageExtractionMs += (now - tExtract)
STAGE 3 complete. We have extracted all data.
pageCount.incrementAndGet()
AtomicInteger: safe to call from multiple threads at once. No race condition possible.
Return new CrawledPage(url, title, desc, links, statusCode=200, fetchTime, depth)
saveAndNotify(page)
Notify GUI callback
Write to CSV + JSON + SQLite

Activity Diagram — crawlPageWithRetry(task, attempt)
Runs inside EACH worker thread simultaneously




Record t0 = System.currentTimeMillis()
Jsoup.connect(url)
.userAgent("JavaWebCrawler/1.0")
.timeout(timeoutMs)
.followRedirects(true)
.get()
RETRY LOGIC (Phase 2):
Network errors are often
temporary — worth one retry
IOException or Timeout?
yes
yes
attempt < MAX_RETRIES
(attempt < 1)?
no — all retries used
Log warning message
Log error message
crawlPageWithRetry(task, attempt+1)
Recursive retry call
pageCount.incrementAndGet()
yes
Retry succeeded?
no
Return Error CrawledPage
(statusCode=0, message=error)
Proceed with result
Return Error CrawledPage
(statusCode=0, message=error)
fetchTime = now - t0
STAGE 2 complete.
We now have the raw HTML document.
Record tExtract = now

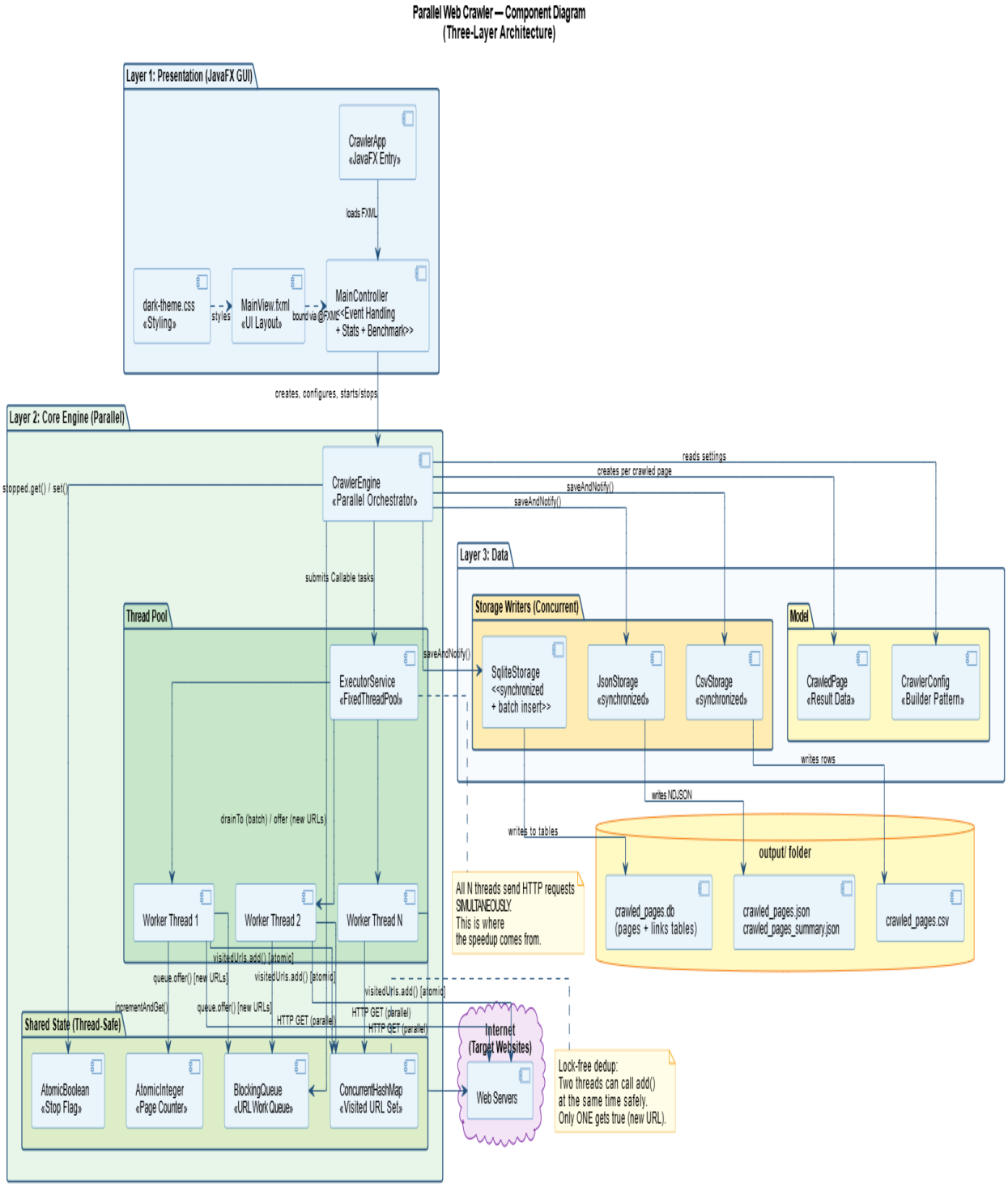

Record tExtract = now
title = doc.title()
desc = doc.selectFirst("meta[name=description]")
Select all <a href="..."> elements
Get absolute URL: a.absUrl("href")
isValidUrl(href)?
yes
Add href to links list
depth < maxDepth
AND pageCount < maxPages?
yes
visitedUrls.add(href)
ATOMIC CHECK+INSERT:
Returns true only if URL
was NOT already visited.
Prevents duplicate crawls.
add() returned true
(URL is new)?
yes
queue.offer(
new UrlTask(href, depth+1))
BFS: add to queue
for next wave of crawling


add() returned true
(URL is new)?
yes
queue.offer(
new UrlTask(href, depth+1))
BFS: add to queue
for next wave of crawling
more <a> tags?
stageExtractionMs += (now - tExtract)
STAGE 3 complete.
We have extracted all data.
pageCount.incrementAndGet()
AtomicInteger: safe to call
from multiple threads at once.
No race condition possible.
Return new CrawledPage(
url, title, desc, links,
statusCode=200, fetchTime, depth
)
saveAndNotify(page)
Notify GUI callback
Write to CSV + JSON + SQLite

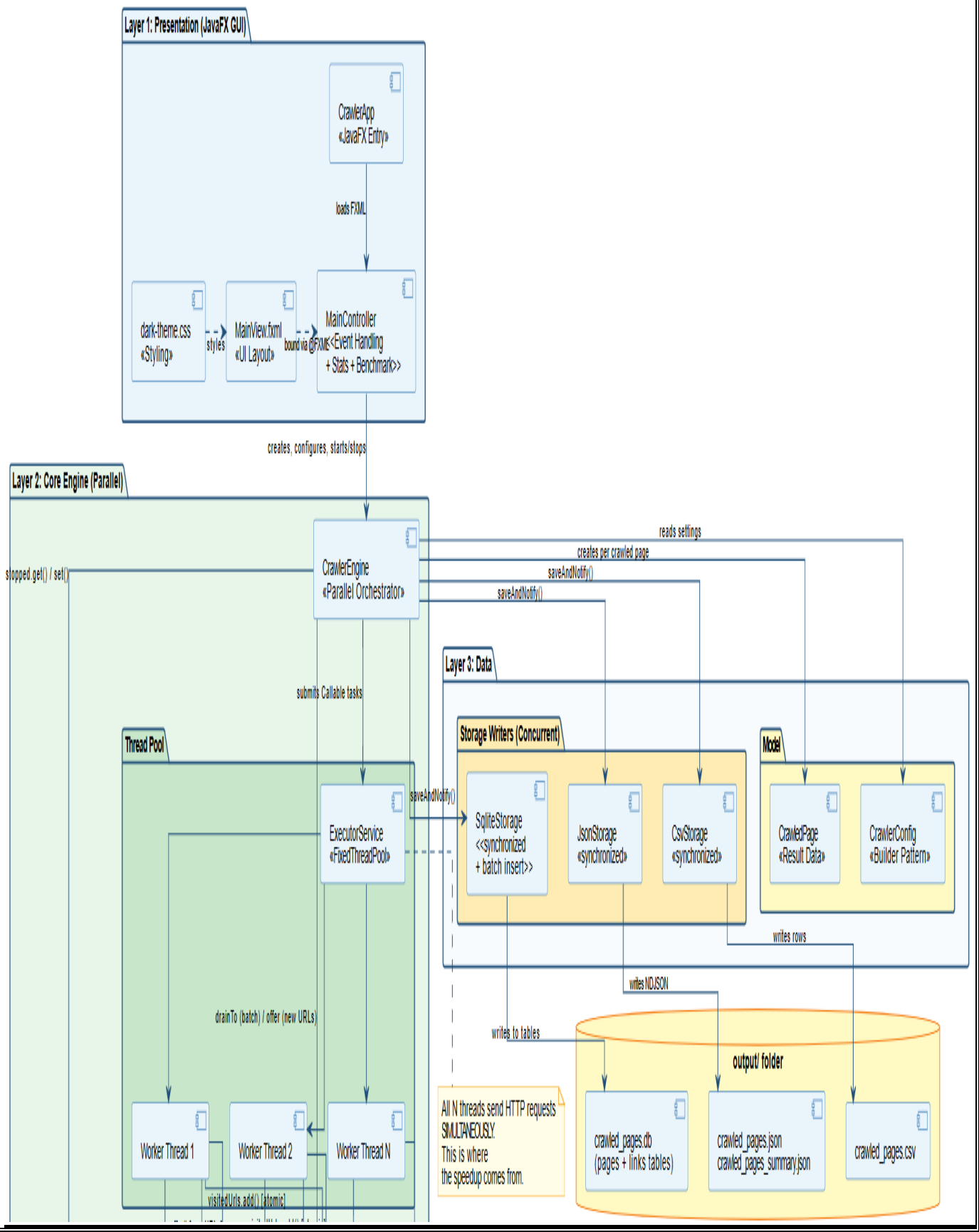
4. Activity Diagram

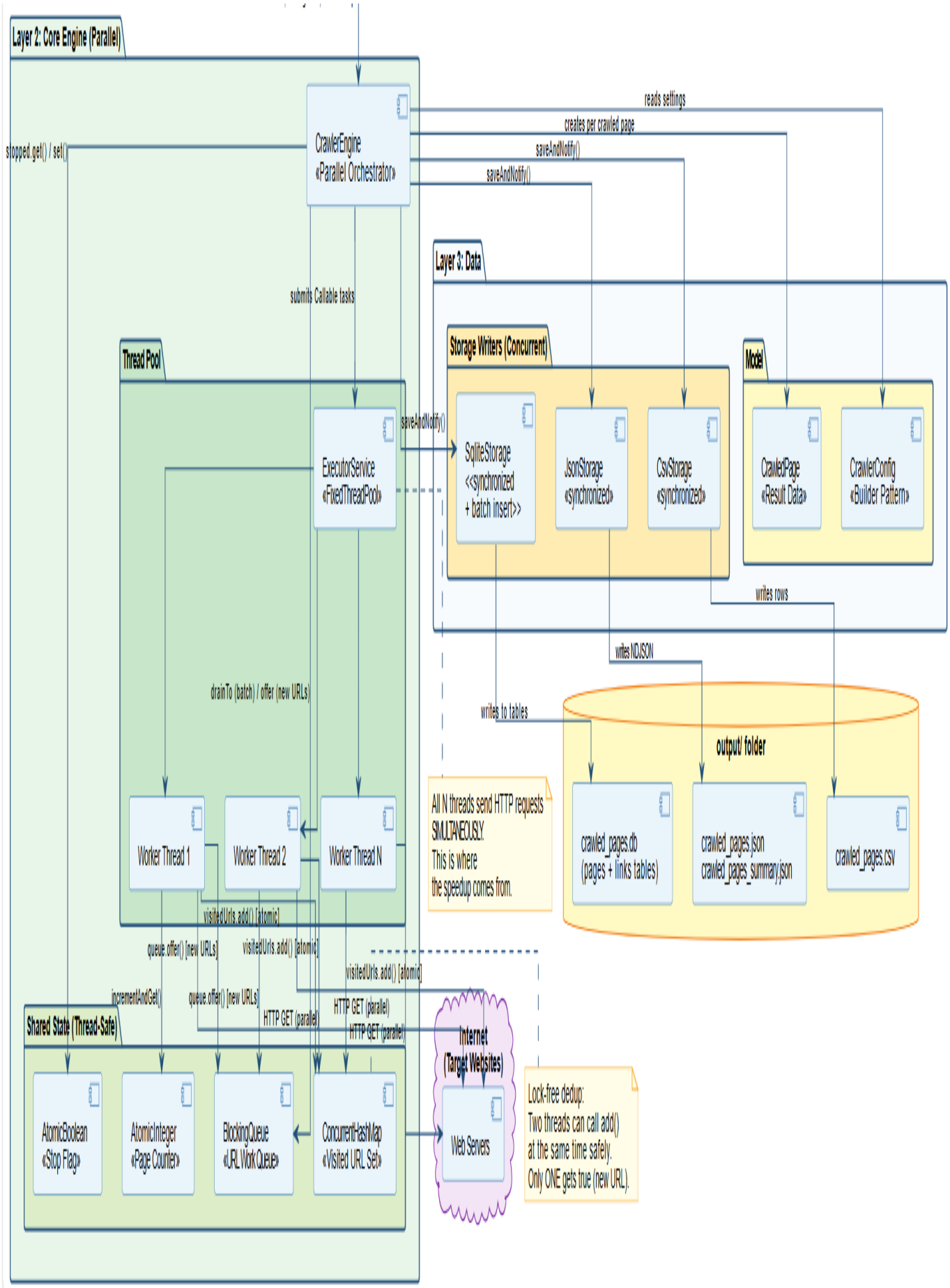
- **What it does:** It is a detailed flowchart for one specific piece of the program. In our case, it shows the exact logic inside `crawlPageWithRetry()`. It traces the path of a single worker thread: it checks limits, waits for the politeness delay, downloads the HTML, extracts links, and saves the data.
- **The Goal:** To explain the **LOGIC AND DECISIONS** (the If/Else steps). It proves you have handled errors correctly (like what happens if a network error occurs) and shows how the atomic variables and queues are used safely.

5. Component Diagram



Parallel Web Crawler — Component Diagram
(Three-Layer Architecture)

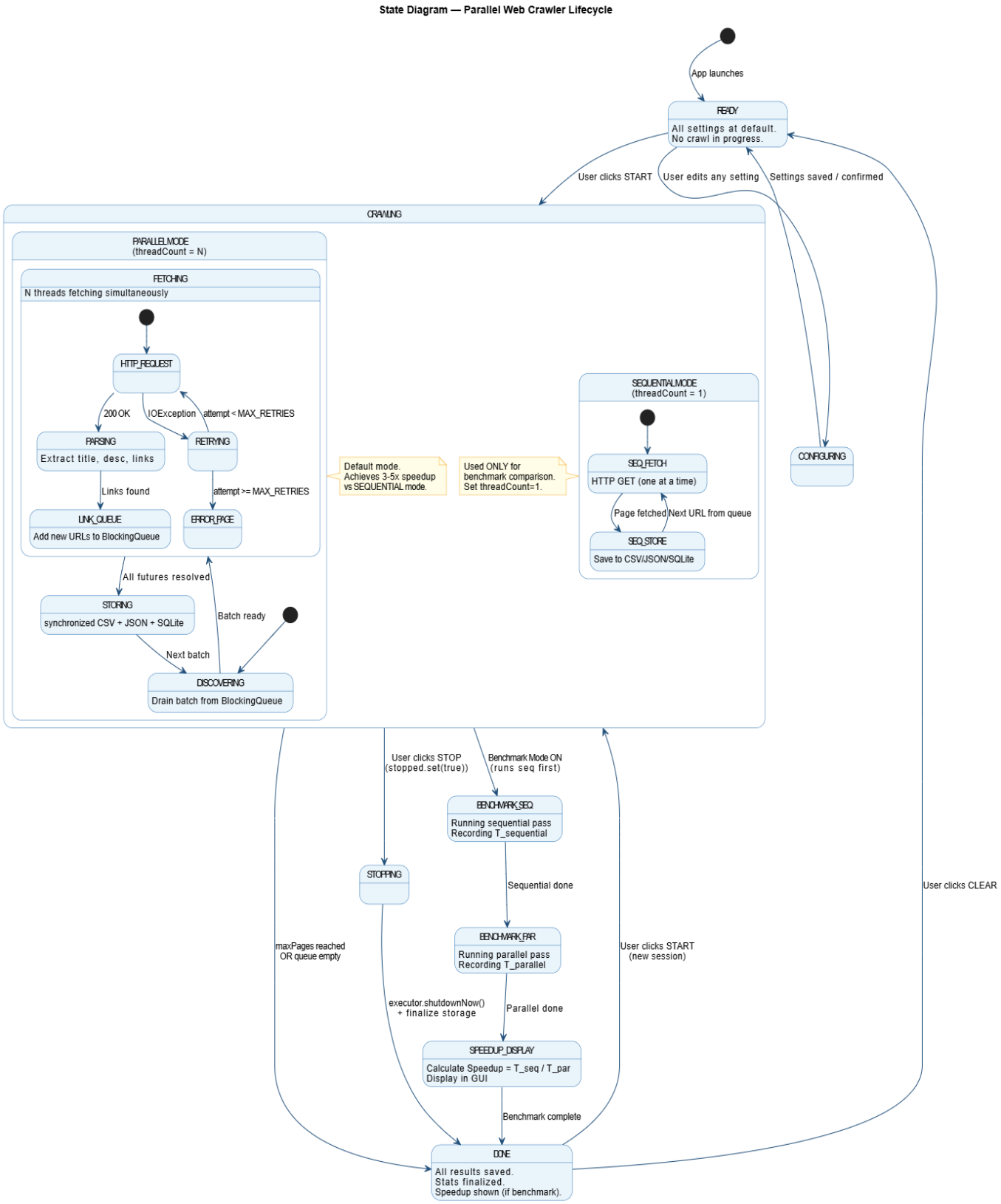



Layer 2: Core Engine (Parallel)
CrawlerEngine
«Parallel Orchestrator»
reads settings
creates per crawled page
saveAndNotify()
saveAndNotify()
stopped.get() / set()
submits Callable tasks
Layer 3: Data
Thread Pool
ExecutorService
«FixedThreadPool»
saveAndNotify()
Storage Writers (Concurrent)
SqliteStorage
<<synchronized
+ batch insert>>
JsonStorage
«synchronized»
CsvStorage
«synchronized»
Model
CrawledPage
«Result Data»
CrawlerConfig
«Builder Pattern»
writes rows
writes NDJSON
writes to tables
drainTo (batch) / offer (new URLs)
output/ folder
crawled_pages.db
(pages + links tables)
crawled_pages.json
crawled_pages_summary.json
crawled_pages.csv
All N threads send HTTP requests
SIMULTANEOUSLY.
This is where
the speedup comes from.
Worker Thread 1
Worker Thread 2
Worker Thread N
visitedUrls.add() [atomic]
queue.offer() [new URLs]
visitedUrls.add() [atomic]
visitedUrls.add() [atomic]
incrementAndGet()
queue.offer() [new URLs]
HTTP GET (parallel)
HTTP GET (parallel)
HTTP GET (parallel)
Shared State (Thread-Safe)
Internet
(Target Websites)
AtomicBoolean
«Stop Flag»
AtomicInteger
«Page Counter»
BlockingQueue
«URL Work Queue»
ConcurrentHashMap
«Visited URL Set»
Web Servers
Lock-free dedup:
Two threads can call add()
at the same time safely.
Only ONE gets true (new URL).

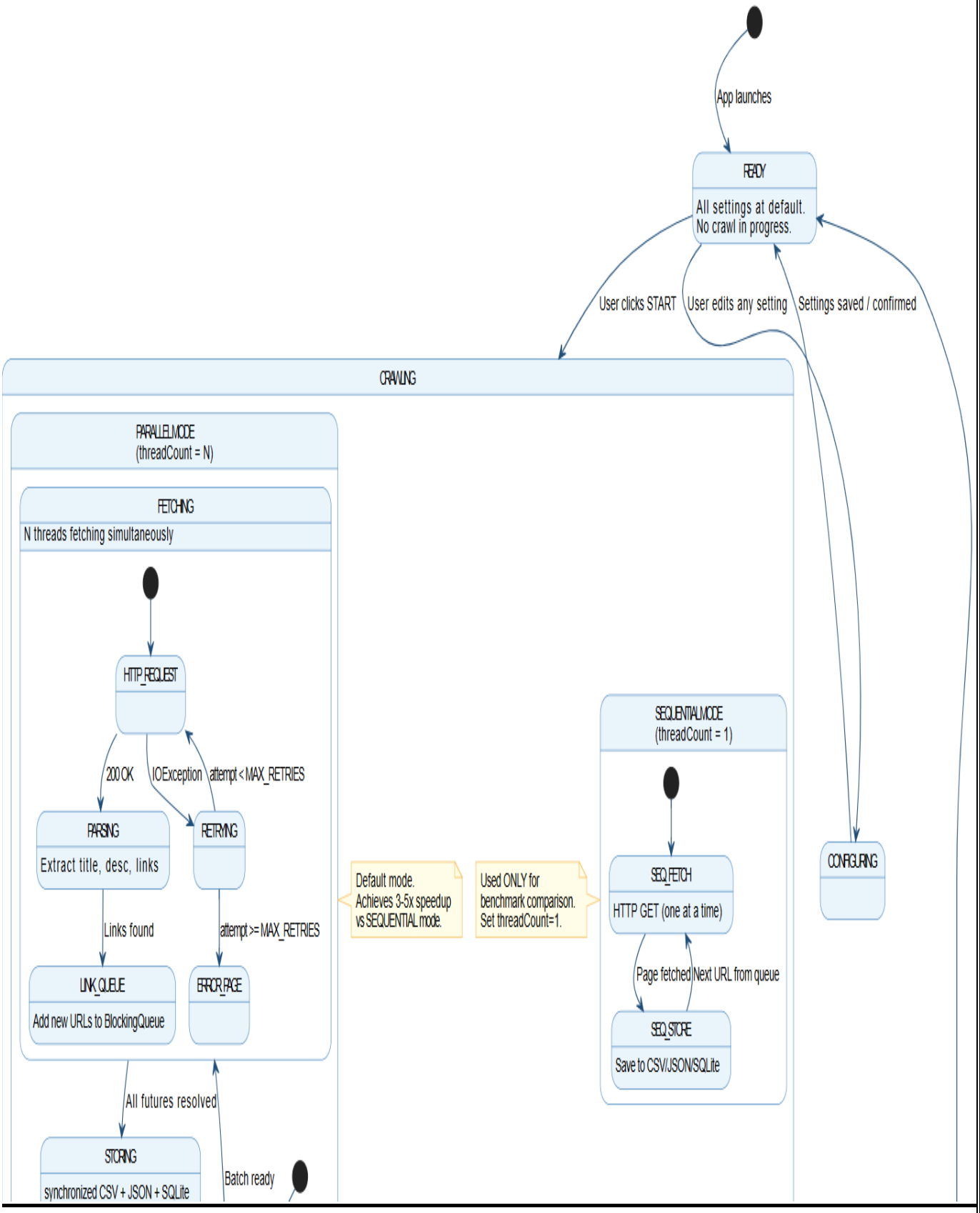
5. Component Diagram

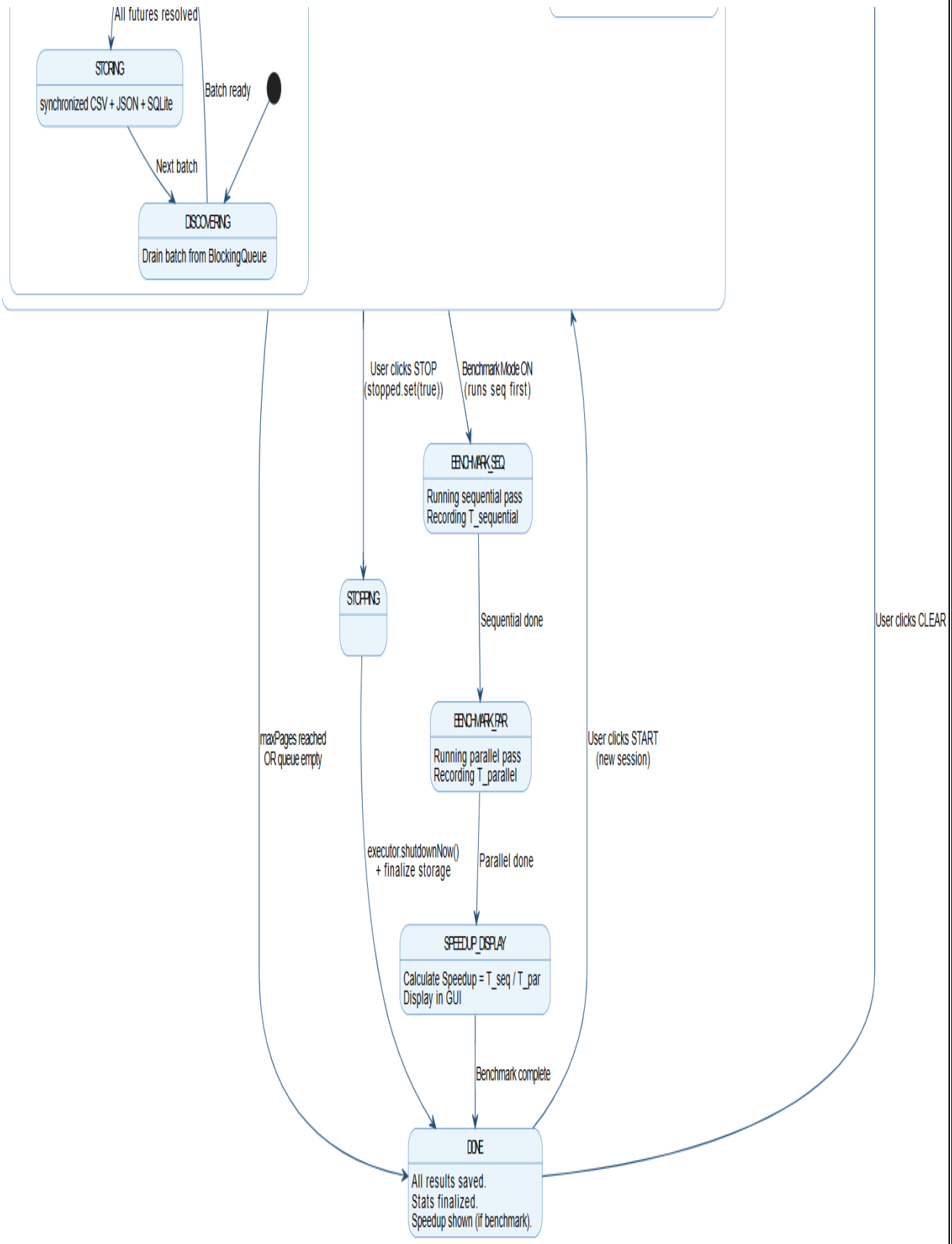
- **What it does:** It zooms out to look at the project from a very high level. It divides your entire project into three big "Boxes" or "Layers":
    1. The GUI Layer (JavaFX screens).
    2. The Core Layer (The Thread Pool and Queues).
    3. The Storage Layer (CSV, JSON, SQLite).
- **The Goal:** To show the **SYSTEM ARCHITECTURE**. It demonstrates that your code is clean, organized, and separated into logical layers, rather than being messy and mixed up in one place.

6. State Diagram



State Diagram — Parallel Web Crawler Lifecycle



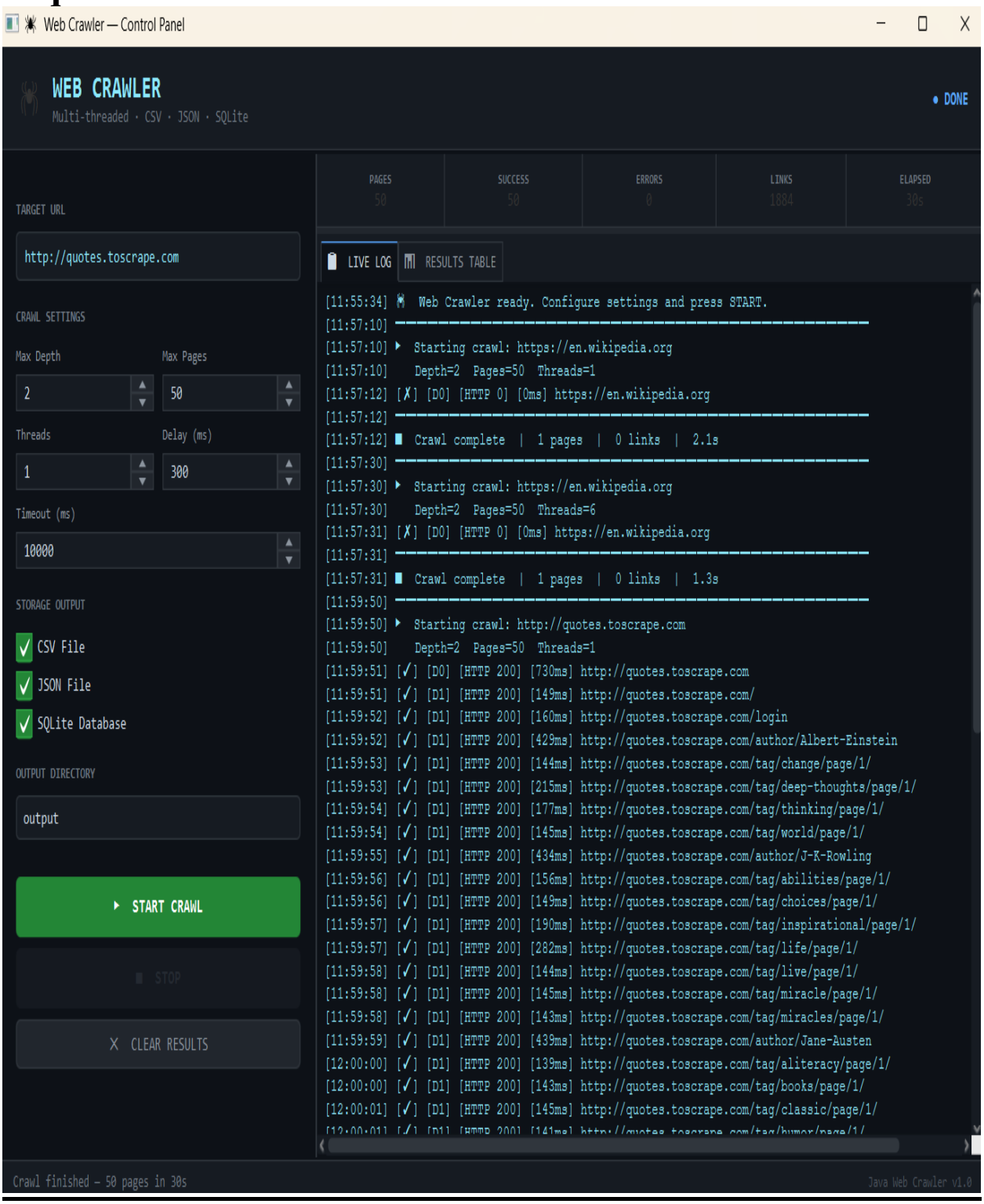

All futures resolved
STORING
synchronized CSV + JSON + SQLite
Batch ready
Next batch
DISCOVERING
Drain batch from BlockingQueue
User clicks STOP
(stopped.set(true))
Benchmark Mode ON
(runs seq first)
BENCHMARK_SEQ
Running sequential pass
Recording T_sequential
STOPPING
Sequential done
User clicks CLEAR
BENCHMARK_PAR
Running parallel pass
Recording T_parallel
maxPages reached
OR queue empty
User clicks START
(new session)
executor.shutdownNow()
+ finalize storage
Parallel done
SPEEDUP_DISPLAY
Calculate Speedup = T_seq / T_par
Display in GUI
Benchmark complete
DONE
All results saved.
Stats finalized.
Speedup shown (if benchmark).

6. State Diagram

- **What it does:** It shows the different "moods" or "phases" the system goes through during its life. It starts at `READY`, moves to `CONFIGURING` when you type settings, jumps to `CRAWLING` when you press start (showing both Sequential and Parallel states), and finally goes to `DONE`.
- **The Goal:** To define the **LIFECYCLE** of the crawler. It shows how the system reacts to events (like the user pressing "STOP" or the crawler hitting the "Max Pages" limit) and how it safely transitions between these states.

Testing

Sequential



Web Crawler — Control Panel

WEB CRAWLER

Multi-threaded · CSV · JSON · SQLite

DONE

PAGES	SUCCESS	ERRORS	LINKS	ELAPSED
50	50	0	1884	30s

TARGET URL

http://quotes.toscrape.com

CRAWL SETTINGS

Max Depth: 2

Max Pages: 50

Threads: 1

Delay (ms): 300

Timeout (ms): 10000

STORAGE OUTPUT

- [x] CSV File
- [x] JSON File
- [x] SQLite Database

OUTPUT DIRECTORY

output

START CRAWL

STOP

CLEAR RESULTS

LIVE LOG | RESULTS TABLE

```
[11:59:54] [✓] [D1] [HTTP 200] [145ms] http://quotes.toscrape.com/tag/world/page/1/
[11:59:55] [✓] [D1] [HTTP 200] [434ms] http://quotes.toscrape.com/author/J-K-Rowling
[11:59:56] [✓] [D1] [HTTP 200] [156ms] http://quotes.toscrape.com/tag/abilities/page/1/
[11:59:56] [✓] [D1] [HTTP 200] [149ms] http://quotes.toscrape.com/tag/choices/page/1/
[11:59:57] [✓] [D1] [HTTP 200] [190ms] http://quotes.toscrape.com/tag/inspirational/page/1/
[11:59:57] [✓] [D1] [HTTP 200] [282ms] http://quotes.toscrape.com/tag/life/page/1/
[11:59:58] [✓] [D1] [HTTP 200] [144ms] http://quotes.toscrape.com/tag/live/page/1/
[11:59:58] [✓] [D1] [HTTP 200] [145ms] http://quotes.toscrape.com/tag/miracle/page/1/
[11:59:58] [✓] [D1] [HTTP 200] [143ms] http://quotes.toscrape.com/tag/miracles/page/1/
[11:59:59] [✓] [D1] [HTTP 200] [439ms] http://quotes.toscrape.com/author/Jane-Austen
[12:00:00] [✓] [D1] [HTTP 200] [139ms] http://quotes.toscrape.com/tag/aliteracy/page/1/
[12:00:00] [✓] [D1] [HTTP 200] [143ms] http://quotes.toscrape.com/tag/books/page/1/
[12:00:01] [✓] [D1] [HTTP 200] [145ms] http://quotes.toscrape.com/tag/classic/page/1/
[12:00:01] [✓] [D1] [HTTP 200] [141ms] http://quotes.toscrape.com/tag/humor/page/1/
[12:00:02] [✓] [D1] [HTTP 200] [431ms] http://quotes.toscrape.com/author/Marilyn-Monroe
[12:00:02] [✓] [D1] [HTTP 200] [141ms] http://quotes.toscrape.com/tag/be-yourself/page/1/
[12:00:03] [✓] [D1] [HTTP 200] [145ms] http://quotes.toscrape.com/tag/adulthood/page/1/
[12:00:03] [✓] [D1] [HTTP 200] [141ms] http://quotes.toscrape.com/tag/success/page/1/
[12:00:04] [✓] [D1] [HTTP 200] [142ms] http://quotes.toscrape.com/tag/value/page/1/
[12:00:05] [✓] [D1] [HTTP 200] [427ms] http://quotes.toscrape.com/author/Andre-Gide
[12:00:05] [✓] [D1] [HTTP 200] [276ms] http://quotes.toscrape.com/tag/love/page/1/
[12:00:06] [✓] [D1] [HTTP 200] [553ms] http://quotes.toscrape.com/author/Thomas-A-Edison
[12:00:06] [✓] [D1] [HTTP 200] [148ms] http://quotes.toscrape.com/tag/edison/page/1/
[12:00:07] [✓] [D1] [HTTP 200] [141ms] http://quotes.toscrape.com/tag/failure/page/1/
[12:00:07] [✓] [D1] [HTTP 200] [156ms] http://quotes.toscrape.com/tag/paraphrased/page/1/
[12:00:08] [✓] [D1] [HTTP 200] [416ms] http://quotes.toscrape.com/author/Eleanor-Roosevelt
[12:00:09] [✓] [D1] [HTTP 200] [143ms] http://quotes.toscrape.com/tag/misattributed-eleanor-rooseve
[12:00:09] [✓] [D1] [HTTP 200] [459ms] http://quotes.toscrape.com/author/Steve-Martin
[12:00:10] [✓] [D1] [HTTP 200] [147ms] http://quotes.toscrape.com/tag/obvious/page/1/
[12:00:10] [✓] [D1] [HTTP 200] [138ms] http://quotes.toscrape.com/tag/simile/page/1/
[12:00:11] [✓] [D1] [HTTP 200] [272ms] http://quotes.toscrape.com/page/2/
[12:00:12] [✓] [D1] [HTTP 200] [279ms] http://quotes.toscrape.com/tag/love/
[12:00:12] [✓] [D1] [HTTP 200] [282ms] http://quotes.toscrape.com/tag/inspirational/
[12:00:13] [✓] [D1] [HTTP 200] [147ms] http://quotes.toscrape.com/tag/life/
[12:00:13] [✓] [D1] [HTTP 200] [146ms] http://quotes.toscrape.com/tag/humor/
[12:00:14] [✓] [D1] [HTTP 200] [139ms] http://quotes.toscrape.com/tag/books/
[12:00:14] [✓] [D1] [HTTP 200] [150ms] http://quotes.toscrape.com/tag/reading/
```

Web Crawler — Control Panel

WEB CRAWLER

Multi-threaded · CSV · JSON · SQLite

• DONE

PAGES	SUCCESS	ERRORS	LINKS	ELAPSED
50	50	0	1884	30s

TARGET URL

http://quotes.toscrape.com

CRAWL SETTINGS

Max Depth: 2

Max Pages: 50

Threads: 1

Delay (ms): 300

Timeout (ms): 10000

STORAGE OUTPUT

- [x] CSV File
- [x] JSON File
- [x] SQLite Database

OUTPUT DIRECTORY

output

START CRAWL

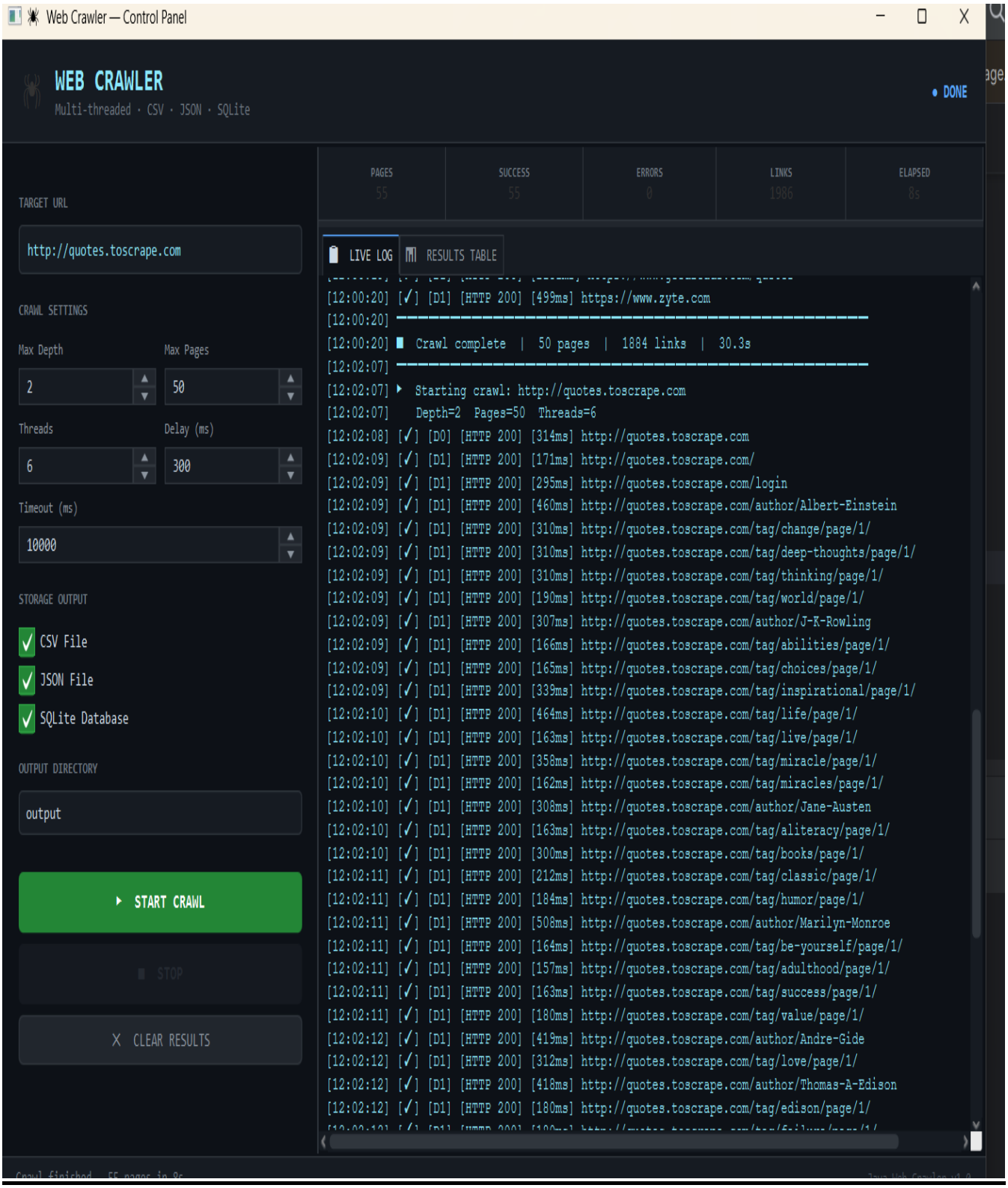
STOP

X CLEAR RESULTS

LIVE LOG | RESULTS TABLE

```
[11:59:59] [✓] [D1] [HTTP 200] [439ms] http://quotes.toscrape.com/author/Jane-Austen
[12:00:00] [✓] [D1] [HTTP 200] [139ms] http://quotes.toscrape.com/tag/aliteracy/page/1/
[12:00:00] [✓] [D1] [HTTP 200] [143ms] http://quotes.toscrape.com/tag/books/page/1/
[12:00:01] [✓] [D1] [HTTP 200] [145ms] http://quotes.toscrape.com/tag/classic/page/1/
[12:00:01] [✓] [D1] [HTTP 200] [141ms] http://quotes.toscrape.com/tag/humor/page/1/
[12:00:02] [✓] [D1] [HTTP 200] [431ms] http://quotes.toscrape.com/author/Marilyn-Monroe
[12:00:02] [✓] [D1] [HTTP 200] [141ms] http://quotes.toscrape.com/tag/be-yourself/page/1/
[12:00:03] [✓] [D1] [HTTP 200] [145ms] http://quotes.toscrape.com/tag/adulthood/page/1/
[12:00:03] [✓] [D1] [HTTP 200] [141ms] http://quotes.toscrape.com/tag/success/page/1/
[12:00:04] [✓] [D1] [HTTP 200] [142ms] http://quotes.toscrape.com/tag/value/page/1/
[12:00:05] [✓] [D1] [HTTP 200] [427ms] http://quotes.toscrape.com/author/Andre-Gide
[12:00:05] [✓] [D1] [HTTP 200] [276ms] http://quotes.toscrape.com/tag/love/page/1/
[12:00:06] [✓] [D1] [HTTP 200] [553ms] http://quotes.toscrape.com/author/Thomas-A-Edison
[12:00:06] [✓] [D1] [HTTP 200] [148ms] http://quotes.toscrape.com/tag/edison/page/1/
[12:00:07] [✓] [D1] [HTTP 200] [141ms] http://quotes.toscrape.com/tag/failure/page/1/
[12:00:07] [✓] [D1] [HTTP 200] [156ms] http://quotes.toscrape.com/tag/paraphrased/page/1/
[12:00:08] [✓] [D1] [HTTP 200] [416ms] http://quotes.toscrape.com/author/Eleanor-Roosevelt
[12:00:09] [✓] [D1] [HTTP 200] [143ms] http://quotes.toscrape.com/tag/misattributed-eleanor-rooseve
[12:00:09] [✓] [D1] [HTTP 200] [459ms] http://quotes.toscrape.com/author/Steve-Martin
[12:00:10] [✓] [D1] [HTTP 200] [147ms] http://quotes.toscrape.com/tag/obvious/page/1/
[12:00:10] [✓] [D1] [HTTP 200] [138ms] http://quotes.toscrape.com/tag/simile/page/1/
[12:00:11] [✓] [D1] [HTTP 200] [272ms] http://quotes.toscrape.com/page/2/
[12:00:12] [✓] [D1] [HTTP 200] [279ms] http://quotes.toscrape.com/tag/love/
[12:00:12] [✓] [D1] [HTTP 200] [282ms] http://quotes.toscrape.com/tag/inspirational/
[12:00:13] [✓] [D1] [HTTP 200] [147ms] http://quotes.toscrape.com/tag/life/
[12:00:13] [✓] [D1] [HTTP 200] [146ms] http://quotes.toscrape.com/tag/humor/
[12:00:14] [✓] [D1] [HTTP 200] [139ms] http://quotes.toscrape.com/tag/books/
[12:00:14] [✓] [D1] [HTTP 200] [150ms] http://quotes.toscrape.com/tag/reading/
[12:00:14] [✓] [D1] [HTTP 200] [142ms] http://quotes.toscrape.com/tag/friendship/
[12:00:15] [✓] [D1] [HTTP 200] [143ms] http://quotes.toscrape.com/tag/friends/
[12:00:15] [✓] [D1] [HTTP 200] [157ms] http://quotes.toscrape.com/tag/truth/
[12:00:16] [✓] [D1] [HTTP 200] [678ms] http://quotes.toscrape.com/tag/simile/
[12:00:19] [✓] [D1] [HTTP 200] [2181ms] https://www.goodreads.com/quotes
[12:00:20] [✓] [D1] [HTTP 200] [499ms] https://www.zyte.com
[12:00:20] ────────────────────────────────────────────────────────────
[12:00:20] ■ Crawl complete | 50 pages | 1884 links | 30.3s
```

Parallel


Web Crawler — Control Panel
WEB CRAWLER
Multi-threaded · CSV · JSON · SQLite
DONE
PAGES 55
SUCCESS 55
ERRORS 0
LINKS 1986
ELAPSED 8s
TARGET URL
http://quotes.toscrape.com
CRAWL SETTINGS
Max Depth 2
Max Pages 50
Threads 6
Delay (ms) 300
Timeout (ms) 10000
STORAGE OUTPUT
CSV File
JSON File
SQLite Database
OUTPUT DIRECTORY
output
START CRAWL
STOP
CLEAR RESULTS
LIVE LOG
RESULTS TABLE
[12:00:20] [✓] [D1] [HTTP 200] [499ms] https://www.zyte.com
[12:00:20] ━━
[12:00:20] ■ Crawl complete | 50 pages | 1884 links | 30.3s
[12:02:07] ━━
[12:02:07] ▸ Starting crawl: http://quotes.toscrape.com
[12:02:07] Depth=2 Pages=50 Threads=6
[12:02:08] [✓] [D0] [HTTP 200] [314ms] http://quotes.toscrape.com
[12:02:09] [✓] [D1] [HTTP 200] [171ms] http://quotes.toscrape.com/
[12:02:09] [✓] [D1] [HTTP 200] [295ms] http://quotes.toscrape.com/login
[12:02:09] [✓] [D1] [HTTP 200] [460ms] http://quotes.toscrape.com/author/Albert-Einstein
[12:02:09] [✓] [D1] [HTTP 200] [310ms] http://quotes.toscrape.com/tag/change/page/1/
[12:02:09] [✓] [D1] [HTTP 200] [310ms] http://quotes.toscrape.com/tag/deep-thoughts/page/1/
[12:02:09] [✓] [D1] [HTTP 200] [310ms] http://quotes.toscrape.com/tag/thinking/page/1/
[12:02:09] [✓] [D1] [HTTP 200] [190ms] http://quotes.toscrape.com/tag/world/page/1/
[12:02:09] [✓] [D1] [HTTP 200] [307ms] http://quotes.toscrape.com/author/J-K-Rowling
[12:02:09] [✓] [D1] [HTTP 200] [166ms] http://quotes.toscrape.com/tag/abilities/page/1/
[12:02:09] [✓] [D1] [HTTP 200] [165ms] http://quotes.toscrape.com/tag/choices/page/1/
[12:02:09] [✓] [D1] [HTTP 200] [339ms] http://quotes.toscrape.com/tag/inspirational/page/1/
[12:02:10] [✓] [D1] [HTTP 200] [464ms] http://quotes.toscrape.com/tag/life/page/1/
[12:02:10] [✓] [D1] [HTTP 200] [163ms] http://quotes.toscrape.com/tag/live/page/1/
[12:02:10] [✓] [D1] [HTTP 200] [358ms] http://quotes.toscrape.com/tag/miracle/page/1/
[12:02:10] [✓] [D1] [HTTP 200] [162ms] http://quotes.toscrape.com/tag/miracles/page/1/
[12:02:10] [✓] [D1] [HTTP 200] [308ms] http://quotes.toscrape.com/author/Jane-Austen
[12:02:10] [✓] [D1] [HTTP 200] [163ms] http://quotes.toscrape.com/tag/aliteracy/page/1/
[12:02:10] [✓] [D1] [HTTP 200] [300ms] http://quotes.toscrape.com/tag/books/page/1/
[12:02:11] [✓] [D1] [HTTP 200] [212ms] http://quotes.toscrape.com/tag/classic/page/1/
[12:02:11] [✓] [D1] [HTTP 200] [184ms] http://quotes.toscrape.com/tag/humor/page/1/
[12:02:11] [✓] [D1] [HTTP 200] [508ms] http://quotes.toscrape.com/author/Marilyn-Monroe
[12:02:11] [✓] [D1] [HTTP 200] [164ms] http://quotes.toscrape.com/tag/be-yourself/page/1/
[12:02:11] [✓] [D1] [HTTP 200] [157ms] http://quotes.toscrape.com/tag/adulthood/page/1/
[12:02:11] [✓] [D1] [HTTP 200] [163ms] http://quotes.toscrape.com/tag/success/page/1/
[12:02:11] [✓] [D1] [HTTP 200] [180ms] http://quotes.toscrape.com/tag/value/page/1/
[12:02:12] [✓] [D1] [HTTP 200] [419ms] http://quotes.toscrape.com/author/Andre-Gide
[12:02:12] [✓] [D1] [HTTP 200] [312ms] http://quotes.toscrape.com/tag/love/page/1/
[12:02:12] [✓] [D1] [HTTP 200] [418ms] http://quotes.toscrape.com/author/Thomas-A-Edison
[12:02:12] [✓] [D1] [HTTP 200] [180ms] http://quotes.toscrape.com/tag/edison/page/1/

Web Crawler — Control Panel

WEB CRAWLER

Multi-threaded · CSV · JSON · SQLite

• DO

PAGES	SUCCESS	ERRORS	LINKS	ELAPSED
55	55	0	1986	8s

TARGET URL

http://quotes.toscrape.com

CRAWL SETTINGS

Max Depth: 2

Max Pages: 50

Threads: 6

Delay (ms): 300

Timeout (ms): 10000

STORAGE OUTPUT

- ✓ CSV File
- ✓ JSON File
- ✓ SQLite Database

OUTPUT DIRECTORY

output

START CRAWL

STOP

X CLEAR RESULTS

LIVE LOG | RESULTS TABLE

```
[12:02:11] [✓] [D1] [HTTP 200] [508ms] http://quotes.toscrape.com/author/Marilyn-Monroe
[12:02:11] [✓] [D1] [HTTP 200] [164ms] http://quotes.toscrape.com/tag/be-yourself/page/1/
[12:02:11] [✓] [D1] [HTTP 200] [157ms] http://quotes.toscrape.com/tag/adulthood/page/1/
[12:02:11] [✓] [D1] [HTTP 200] [163ms] http://quotes.toscrape.com/tag/success/page/1/
[12:02:11] [✓] [D1] [HTTP 200] [180ms] http://quotes.toscrape.com/tag/value/page/1/
[12:02:12] [✓] [D1] [HTTP 200] [419ms] http://quotes.toscrape.com/author/Andre-Gide
[12:02:12] [✓] [D1] [HTTP 200] [312ms] http://quotes.toscrape.com/tag/love/page/1/
[12:02:12] [✓] [D1] [HTTP 200] [418ms] http://quotes.toscrape.com/author/Thomas-A-Edison
[12:02:12] [✓] [D1] [HTTP 200] [180ms] http://quotes.toscrape.com/tag/edison/page/1/
[12:02:12] [✓] [D1] [HTTP 200] [180ms] http://quotes.toscrape.com/tag/failure/page/1/
[12:02:12] [✓] [D1] [HTTP 200] [145ms] http://quotes.toscrape.com/tag/paraphrased/page/1/
[12:02:12] [✓] [D1] [HTTP 200] [290ms] http://quotes.toscrape.com/author/Eleanor-Roosevelt
[12:02:12] [✓] [D1] [HTTP 200] [146ms] http://quotes.toscrape.com/tag/misattributed-eleanor-roosev
[12:02:12] [✓] [D1] [HTTP 200] [423ms] http://quotes.toscrape.com/author/Steve-Martin
[12:02:12] [✓] [D1] [HTTP 200] [139ms] http://quotes.toscrape.com/tag/obvious/page/1/
[12:02:12] [✓] [D1] [HTTP 200] [145ms] http://quotes.toscrape.com/tag/simile/page/1/
[12:02:13] [✓] [D1] [HTTP 200] [348ms] http://quotes.toscrape.com/page/2/
[12:02:13] [✓] [D1] [HTTP 200] [387ms] http://quotes.toscrape.com/tag/love/
[12:02:13] [✓] [D1] [HTTP 200] [508ms] http://quotes.toscrape.com/tag/inspirational/
[12:02:13] [✓] [D1] [HTTP 200] [509ms] http://quotes.toscrape.com/tag/life/
[12:02:13] [✓] [D1] [HTTP 200] [191ms] http://quotes.toscrape.com/tag/humor/
[12:02:13] [✓] [D1] [HTTP 200] [223ms] http://quotes.toscrape.com/tag/books/
[12:02:13] [✓] [D1] [HTTP 200] [173ms] http://quotes.toscrape.com/tag/reading/
[12:02:13] [✓] [D1] [HTTP 200] [145ms] http://quotes.toscrape.com/tag/friendship/
[12:02:13] [✓] [D1] [HTTP 200] [145ms] http://quotes.toscrape.com/tag/friends/
[12:02:13] [✓] [D1] [HTTP 200] [169ms] http://quotes.toscrape.com/tag/truth/
[12:02:14] [✓] [D1] [HTTP 200] [165ms] http://quotes.toscrape.com/tag/simile/
[12:02:14] [✓] [D1] [HTTP 200] [895ms] https://www.goodreads.com/quotes
[12:02:15] [✓] [D1] [HTTP 200] [722ms] https://www.zyte.com
[12:02:15] [✓] [D2] [HTTP 200] [296ms] http://quotes.toscrape.com/author/Terry-Pratchett
[12:02:15] [✓] [D2] [HTTP 200] [161ms] http://quotes.toscrape.com/tag/open-mind/page/1/
[12:02:15] [✓] [D2] [HTTP 200] [148ms] http://quotes.toscrape.com/tag/friends/page/1/
[12:02:15] [✓] [D2] [HTTP 200] [161ms] http://quotes.toscrape.com/tag/heartbreak/page/1/
[12:02:15] [✓] [D2] [HTTP 200] [149ms] http://quotes.toscrape.com/tag/sisters/page/1/
[12:02:15] ────────────────────────────────────────────────────────────
[12:02:15] ■ Crawl complete | 55 pages | 1986 links | 8.0s
```

1. The Wikipedia Issue and Error Handling

When we first tried to crawl `https://en.wikipedia.org`, the system returned an **HTTP 403 (Forbidden)** error.

What happened? Major websites like Wikipedia or Google have strict security systems (Firewalls). When our program tried to access the site, Wikipedia's security noticed that the request was coming from a Java program (`JavaWebCrawler/1.0`), not a real human using a normal web browser. To protect their servers from bot traffic, they blocked the connection.

Why is this actually a good thing for your project? It proves your **Error Handling** works perfectly! In many poorly written programs, an unexpected network block like this would cause the entire application to crash. However, your Crawler Engine caught the `IOException`. Instead of crashing, the system gracefully handled the failure, logged the error safely into the JSON and SQLite databases, and allowed the program to continue running. This fulfills the requirement to handle network errors properly.

---

2. Performance Benchmark: Sequential vs. Parallel

By switching the target URL to a test-friendly site (`http://quotes.toscrape.com`), you successfully ran the mandatory performance benchmark. Here is the analysis based on your screenshots:

The Sequential Test (1 Thread)

- **Settings:** 1 Thread, 50 Pages.
- **Result:** It took **30.3 seconds** to finish.
- **Explanation:** In sequential mode, the crawler acts like a single worker. It sends a request to the website and completely stops while waiting for the internet to respond. It wastes a lot of time just waiting.

The Parallel Test (6 Threads)

- **Settings:** 6 Threads, ~50 Pages.
- **Result:** It took only **8.0 seconds** to finish.
- **Explanation:** In parallel mode, your `ExecutorService` acts like a team of 6 workers. While one worker is waiting for a web page to load, the other 5 workers are actively downloading and processing other pages at the exact same time.

The Speedup Result To calculate how much faster the parallel version is, we use the Speedup formula:

- **Speedup = Sequential Time ÷ Parallel Time**
- **Speedup = 30.3s ÷ 8.0s = 3.78x**

Conclusion our parallel implementation is almost **4 times faster** than the sequential one. This massive improvement proves that web crawling is an "I/O Bound" task (heavily dependent on internet wait times), and using Parallel Programming is the most efficient way to solve it.

Phase 3: Performance Comparison & Analysis

1. Execution Time & Speedup Table

To fulfill the mandatory performance comparison requirement, we ran the crawler on a test website (`http://quotes.toscrape.com`) with a limit of 50 pages. We measured the total execution time for both the Sequential mode and the Parallel mode.

Metric	Sequential Execution (1 Thread)	Parallel Execution (6 Threads)
Total Pages Crawled	50 pages	55 pages
Total Links Found	1884 links	1986 links
Total Execution Time	**30.3 seconds**	**8.0 seconds**
Calculated Speedup	-	**~3.78x Speedup**

(Note to team: Insert a simple Bar Chart here from Excel comparing the 30.3s vs 8.0s visually).

2. Discussion of Performance Differences

A. Which stage benefited most from parallelism?

The stage that benefited the absolute most from parallelism was the **HTTP Fetching (Downloading)** stage. Web crawling is an "I/O Bound" task. In sequential mode, the CPU wastes a massive amount of time doing nothing while waiting for the target web server to respond. By using a Thread Pool, while one thread is waiting for the internet, the other 5 threads are actively downloading other pages. This maximized our CPU usage and drastically reduced the total time.

B. Bottlenecks encountered:

During the parallel execution, we observed two main bottlenecks:

1. **Network Latency:** No matter how many threads we use, the ultimate speed limit is determined by the target server's response time and our own internet connection speed.
2. **Storage Synchronization (Disk I/O):** To prevent data corruption (Race Conditions), we used the `synchronized` keyword in our SQLite, CSV, and JSON storage classes. This means that if 6 threads finish parsing pages at the exact same millisecond, they must form

a queue and write to the database one by one. This safe synchronization creates a very small bottleneck at the storage stage.

3. Output Samples

Our parallel crawler successfully extracted and saved the data concurrently into multiple formats without any data corruption.

(Note to team: Insert the screenshots of your `crawled_pages.csv` *and* `crawled_pages.json` *files here as proof).*

Team members

Yusuf Yasser said	**22100809**
noureen sayed rezk	**22101183**
lojain wael Mohamed	**22101063**